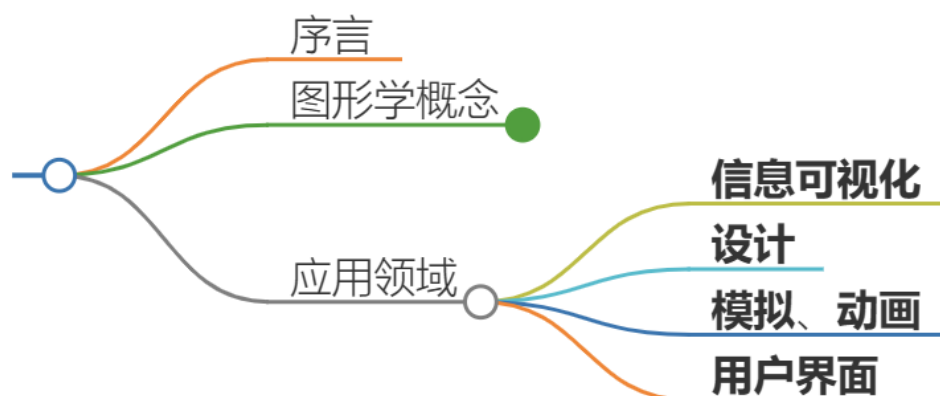


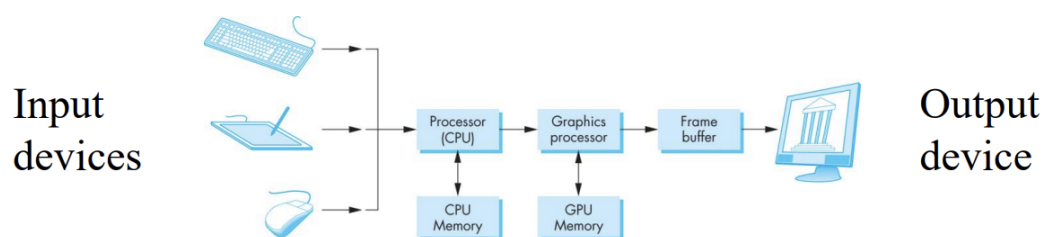
计算机图形学

序言



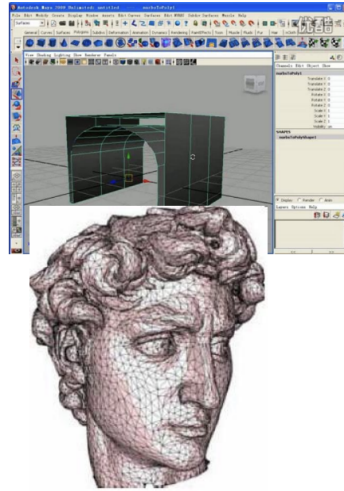
图形学概念

计算机图形学是研究利用计算机生成图像的方法



主要研究内容

- 建模
创建3D几何体



- **渲染**
绘制图片
- **动画**
生成运动物体的视频

应用领域

- 信息可视化
- 设计
- 模拟、动画
- 用户界面

CG和CV

- CG(计算机图形学): 创建图像
- CV(计算机视觉): 解析图像为计算机能够理解的信息

OpenGL介绍



OpenGL是一套可提供与图形系统交互的能力的API，常与C绑定

实现图形接口与实现分离，易于掌握和使用

图形应用程序

图形应用程序具有这样的三要素

- 指定绘制对象
- 描述对象的属性
- 定义对象被观察的方式

对象类型

现代图形系统支持两种对象类型

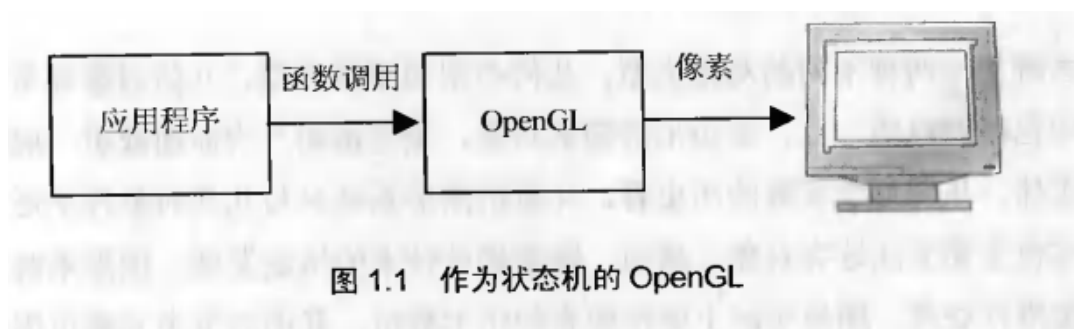
- **几何类型**
包括曲面、直线、多边形这样的几何体
大多数API中，几何对象的定义和显示方式是分离的(绿色的实现和红色的虚线同属于直线类)
- **图像类型**

图像实际上是**像素的矩形数组**，每个元素可用来表示**像素的颜色、灰度和其他属性**

OpenGL介绍

OpenGL状态机

OpenGL可看作是一个具有输入和输出的**状态机**

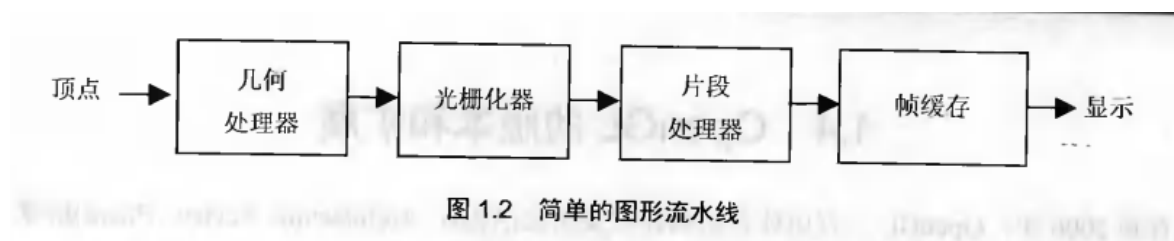


- 应用程序输入**几何对象的三要素**
- OpenGL**处理信息**
- 输出一个像素集合到**显示屏上**

这样，OpenGL的函数就可以被分为**指定输入对象的函数(几何对象的描述信息、离散信息)**和**修改机器状态的函数(修改颜色、观察条件、材质属性)**

状态修改函数包括**指定颜色、观察条件、材质属性**和许多变量的函数，状态决定输入的处理方式

OpenGL绘制流水线



绘制流水线描述了OpenGL是如何给出一个**像素集合**的

1. 接受待绘制形状的**顶点流得到图元**(如多边形)
2. **几何处理器**对图元**实施变换，旋转、平移、缩放**和相对于OpenGL摄像机为图元进行定位
3. **光栅化器**根据**虚拟相机、光源**，生成或渲染二维图像(片段)
4. **片段处理器**负责**上色**，并判断片段是否被其他对象**所阻挡及是否可见**
5. **帧缓存**包含在显示器上将要显示的**像素信息**

OpenGL函数的组成

- **图元函数**

指定要**生成屏幕图像的图元**，这包含**几何图元**(如多边形)和**离散实体**(如位图)

- **属性函数**

控制图元的**外观**，定义了**颜色、线型、材质属性、光照和纹理**

- **观察函数**

指定**摄像机的属性**，可以控制其摄像机的**镜头参数**

- **控制函数**

允许**启用或禁用OpenGL特性**

- **查询函数**

允许**决定某一特定系统的性能、OpenGL状态变量的取值**

- **输入与窗口函数**

不属于OpenGL核心库，但还是被包含在OpenGL的实现如**GLUT库**中

除图元函数外，其他函数都是**修改机器状态的函数**

一些编程约定

- OpenGL函数包含在**gl(核心库)**和**glu(实用库)**中，前者函数以**gl**为前缀，后者函数以**glu**为前缀

- GLUT是OpenGL实用工具包
- OpenGL函数**无法利用面向对象语言**的特性，如**重载**
- 关于类型的约定

OpenGL类型定义	相应C语言类型	后缀定义
GLbyte	char	b
GLubyte, GLboolean	unsigned char	ub
GLshort	short	s
GLushort	unsigned short	us
GLint, GLsizei	int	i
GLuint, GLenum, GLbitfield	unsigned int	ui
GLfloat, GLclampf	float	f
GLdouble, GLclampd	double	d

成像系统



成像系统

图像

物理图像

物理图像即图片，物理成像系统生成物理图像

物理成像系统有

- 照相机
- 显微镜
- 望远镜
- 人类视觉系统

i 矢量图和光栅图

矢量图是由**连接的线**组成的图像，而**光栅图**是由**连接的点**组成的图像



计算机图像

计算机图像的生成过程完全类似于物理成像系统，但来源可以不存在

成像要素

- **对象**

即**几何形状**，在计算机图形学中，通过点、线、多边形等几何图元来构造图像

- **观察者**

观察者观察对象，从而生成对象的图像

- **光源**

光是人能够看到图像的条件

- **材质**

对象表面反射光的属性，可能为**漫反射**、**镜面反射**、**折射**



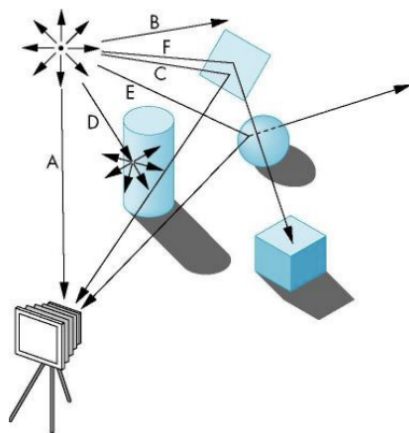
NOTICE

对象和观察者是**相互独立**的，横看成岭侧成峰

跟踪光线

已知**成像要素**，可以通过跟踪光线的手段生成图像

跟踪光线是从**点光源**出发，跟踪**所有的光线**，确定哪些光线进入照相机的镜头



物理成像系统

人类视觉系统

是遵循物理原理的复杂系统。人类通过**感光细胞**感知图像

- **视杆细胞**：仅能感知亮度，不能感知颜色
- **视锥细胞**：感知亮度(较强光)和色彩，有三种类型的锥状细胞，从而只有三种刺激值送到大脑

i 强度和亮度

强度是点光源光线能量的物理度量，**亮度**度量我们所感知到的物体发射出的**光线有多强**

人类对**不同波长光**的响应不同

- 对**强度相同的红光和绿光**，感知到的**亮度不同**
- 对**绿光最敏感**，对**红光和蓝光最不敏感**

三色理论

人类能够区分颜色是由于**三种视锥细胞**具有不同的光谱敏感度。

三色理论是使用**三种基色或原色**来近似所感知到的所有颜色

- **加色系统**

加色系统通过把**三原色的值**加在一起形成一种颜色

原色为**红、绿、蓝**。是**基于自身发光**的颜色系统

- **减色系统**

减色系统通过在白光中**过滤青色、品红色、黄色**形成最终颜色

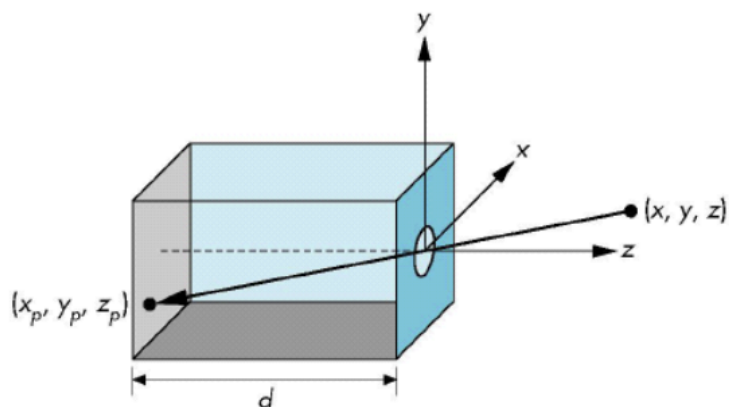
是**基于反光**的颜色系统

针孔照相机

方形盒一个侧面的中心有一个小孔，胶片放在盒子里与小孔相对的内表面上

小孔仅容一条光线进入，胶片平面与针孔距离为d

坐标系原点在针孔处



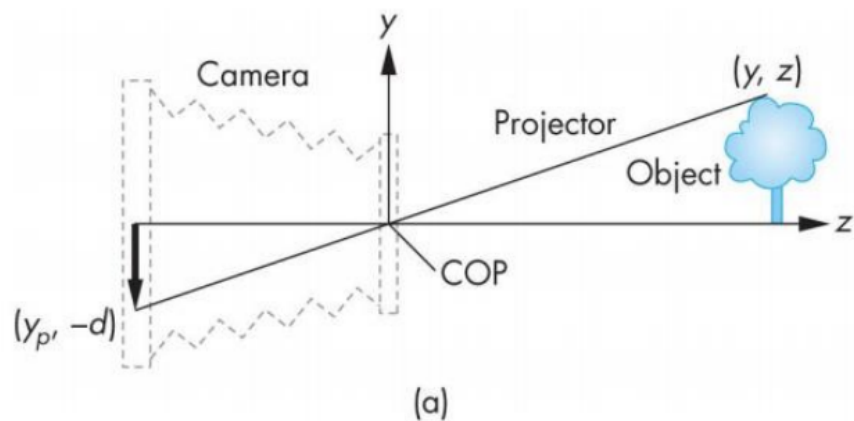
设视角为 θ ，有

$$\theta = 2 \tan^{-1}(h/2d)$$

虚拟照相机系统

虚拟相机基于针孔照相机设计

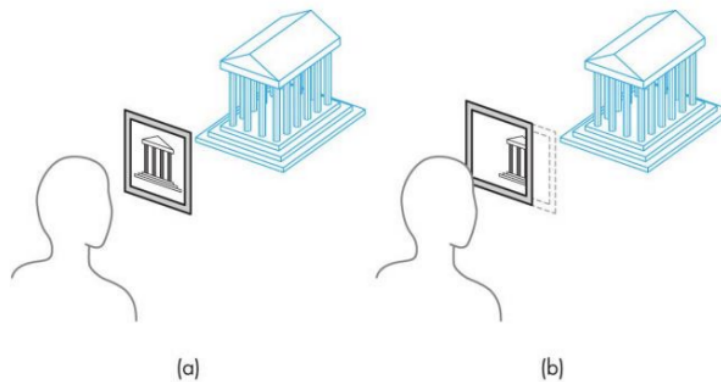
成像模型



- 投影线：对象上一点到透镜中心的连线
- 投影中心：透镜中心

- **投影平面**：移至透镜前的**虚拟成像平面**
- **裁剪窗口**：投影平面的部分

给定**投影中心的位置**、**投影平面的位置和方向**、**裁剪窗口的大小**，就能确定哪些对象会在图像中出现



优势在于

- 把**对象**、**观察者**和**光源**区分开
- 可以得到**简单的软件API**。指定**对象**、**光源**、**照相机**、材料属性就可由API的实现确定最终的图像
- 可以得到**快速的硬件实现**

OpenGL和**Direct3D**均基于该模型

全局与局部光照

• 全局光照

构建一个**场景内模型**，能捕获光在发生反射/折射后的所有路径及终点，目标是**照片级渲染**

有**光线跟踪法**和**辐射度法**，但计算量大，**不适合实时或交互式系统**

• 局部光照

构建**方向光**，目标是**交互式或实时**的图形渲染

采用**Phong光照模型**，**OpenGL**，**Direct3D**均实现了这样的光照

流水线体系

渲染的流水线体系按照应用程序定义对象的先后顺序，依次处理每个对象

顶点处理

对象由**一组图元**组成，而每个图元又包含一组顶点

图元的类型和顶点集定义了场景的几何

顶点处理器将对象的顶点所在的坐标转化为另一坐标系下的坐标，计算顶点的颜色

在这个过程中主要依靠投影，通过结合观察者的位置和三维对象确定二维图像

- **透视投影**：所有**投影线交于投影中心**
- **平行投影**：投影线**平行**，投影中心在**无穷远**，用**投影方向**表示

顶点处理器对**各个顶点的处理相互独立**

图元装配

将顶点组装为几何对象的过程，如**线段**、**多边形**、**曲线**和**曲面**

裁剪

将**不在视景物中的对象**从场景中裁剪

光栅化

光栅化为每个图元生成一组片段，在对象上对顶点的属性进行插值得到片段的属性

片段

片段具有**颜色**和**深度**属性，在**帧缓冲区**中有一个位置

片段处理

确定帧缓冲区中相应像素的颜色，颜色可以由纹理映射确定，也可以由顶点颜色插值得到

INFO.

片段可能被离照相机更近的其它片段挡住

图形系统编程接口

API由生成函数和输入和输出函数组成

INFO.

生成函数定义生成一幅图像所需要的内容，如对象、观察者、光源、材料属性

对象的指定

绝大多数API支持有限的基本几何对象，如点、线、多边形、曲线和曲面等
所有基本形状都是通过空间中的位置或顶点定义

相机的指定

由六个自由度(包括投影中心和方向)、镜头、焦距、胶卷尺寸确定

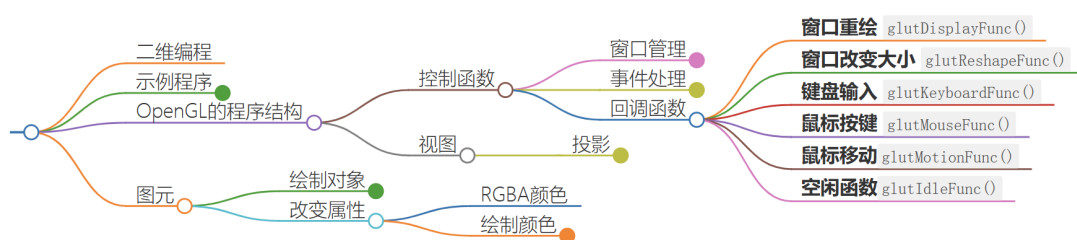
光源

- 点光源与分布式光源
- 聚光灯
- 远光源与近光源
- 光源的颜色属性

材质

- 吸收性：颜色属性
- 反射性：漫反射、镜面

二维编程



示例程序

```
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>          /* glut.h includes gl.h and glu.h*/
#endif

// display回调函数是每个GLUT程序必备的
void display(void){
    /* clear window */

    glClear(GL_COLOR_BUFFER_BIT);// 用于清空帧缓冲区

    /* draw unit square polygon */
```

```

glBegin(GL_POLYGON);    // 图元定义，可选择点、线等
glVertex2f(-0.5, -0.5);
glVertex2f(-0.5, 0.5);
glVertex2f(0.5, 0.5);
glVertex2f(0.5, -0.5);
glEnd();

/* flush GL buffers */

glFlush();

}

int main(int argc, char** argv)
{
    glutInit(&argc, argv); //GLUT初始化，应该在其他GLUT函数和OpenGL
函数之前调用
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); // 显示模式
    glutInitWindowSize(500, 500); // 窗口的大小
    glutInitWindowPosition(0, 0); // 窗口的显示位置
    glutCreateWindow("simple"); // 创建有指定标题名的窗口
    glutDisplayFunc(display); // 这是一个回调函数，调用定义的display
函数(这个函数不能有参数)
    glutMainLoop();          // 事件循环
}

```

OpenGL的程序结构

大部分OpenGL程序像示例程序一样，包含

- **main函数**
main函数打开具有**指定属性**的窗口，进入事件循环
- **init函数**
设置**初始状态变量**，如**视图**、**属性**

- 回调函数

如显示函数display(), 输入和窗口函数

一般先配置开启窗口，初始化OpenGL的状态，再处理用户事件，最终绘制图像

控制函数

控制函数负责与窗口系统通信，处理程序执行期间发生的错误

窗口管理

```
// 初始化GLUT
void glutInit(int * argc, char ** argv)
// 设置窗口的初始宽度和高度，单位为像素，缺省300x300
void glutInitWindowSize(int width, int height)
// 窗口左上角相对于屏幕左上角的位置，单位为像素，缺省(0,0)
void glutInitWindowPosition(int x, int y)
// 设置窗口的显示模式
void glutInitDisplayMode(unsigned int mode)
// 创建窗口，标题为title。调用glutMainLoop()之前，窗口不会显示
int glutCreateWindow(char * title)
```

事件处理

```
// 进入GLUT事件处理循环，当有事件发生时，调用相应的回调函数
void glutMainLoop()
// 每个GLUT程序都必须有的显式回调函数
void glutDisplayFunc(void(*func)(void))
```

回调函数

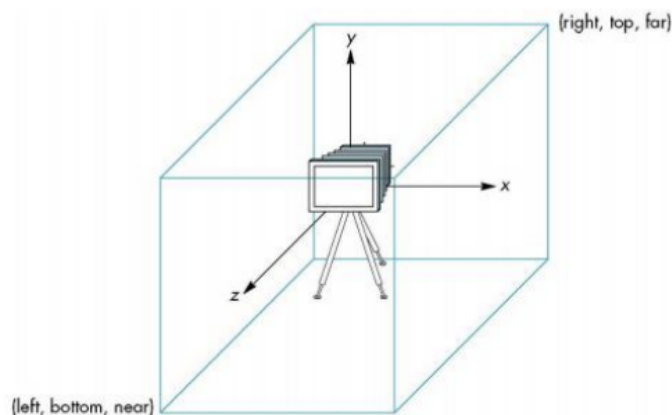
GLUT用回调函数机制来进行事件处理

- 窗口重绘 `glutDisplayFunc()`
- 窗口改变大小 `glutReshapeFunc()`
- 键盘输入 `glutKeyboardFunc()`
- 鼠标按键 `glutMouseFunc()`
- 鼠标移动 `glutMotionFunc()`
- 空闲函数 `glutIdleFunc()`

视图

OpenGL默认视图为**正交投影**，**投影线彼此平行**

相机默认为**对象坐标系的远点**，指向z轴负方向



默认的视景体是一个中心在**原点**，**边长为2的立方体**

投影

投影利用**投影矩阵乘法**进行

首先**设置矩阵类型**

```
void glMatrixMode(GL_PROJECTION);
```

再从单位阵开始，逐渐得到一个**投影矩阵**

```
glLoadIdentity();  
// glOrtho的六个参数分别为左、右、底、顶、近、远  
glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1);  
// 二维状态时可以这样设置正交视景体，相当于裁剪窗口  
gluOrtho2D(GLdouble left, GLdouble right, GLdouble bottom, GLdouble  
top)
```

图元

图元的**顶点坐标**在**对象坐标系**或**世界坐标系**中定义，最终映射成**窗口坐标**或**屏幕坐标**

绘制对象

绘制多边形首先要**指定顶点的位置**，在上面的display函数中首先指定了**正方形的四个顶点**

首先**清空颜色缓存**，避免缓存区干扰接下来绘制图形的显示

```
glClear(GLbitfield mask) //实例中的GL_COLOR_BUFFER_BIT指代颜色缓存
```

声明定义了何种类型的对象

```
/*  
- GL_POINTS 点  
- GL_LINES 线段
```

- GL_LINE_STRIP 开折线
- GL_LINE_LOOP 闭折线
- GL_POLYGON 多边形
- GL_TRIANGLES 三角形
- GL_QUADS 四边形
- GL_TRIANGLE_STRIP 三角形带
- GL_QUAD_STRIP 四边形带
- GL_TRIANGLE_FAN 三角形扇

*/

```
void glBegin(GLenum mode)
```

指定顶点的函数有

```
void glVertex<234><sifd>(x,y,z...) //这里的尖括号代表可能的函数名，如glVertex2f就表示在二维空间内指定坐标为浮点数的顶点的函数
```

```
void glVertex<234><sifd>v(coordinates) //带有v的函数表示接受一个坐标数组，坐标数组的每个元素代表该顶点的分量
```

表示顶点列表声明结束

```
void glEnd()
```

输出缓冲区的结果

```
void glFlush()
```

通常用已有的图元来近似曲线和曲面

- **正n边形来近似圆**
- **三角形和四边形来近似球面**

- **多边形网格来近似曲面**

绘制矩形

```
void glColor(GLenum mode);
```

改变属性

属性包括点的颜色、大小，线段的颜色、宽度、虚实，多边形的绘制模式和填充模式、文本的字体、字号和方向等

RGBA颜色

颜色的每个分量在帧缓冲区中是分开存储的，在缓冲区中通常每个分量占用8位字节

前三者为原色分量，第四个为**不透明度**，为1时透明，0时完全不透明

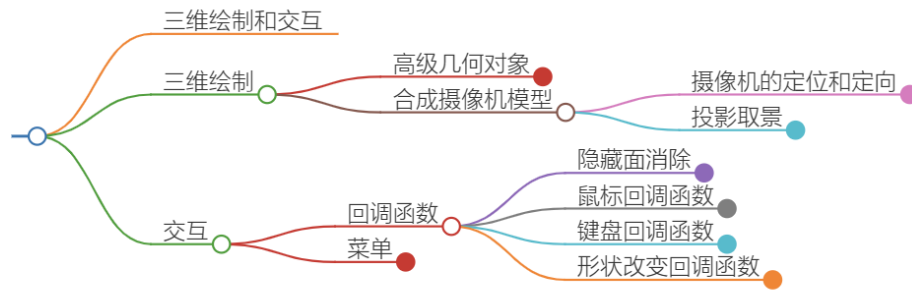
绘制颜色

OpenGL默认使用**平滑着色**，在两个不同颜色的顶点间的颜色由**插值计算**得到，产生**渐变色**

也可以采用**平面着色**，不会产生渐变

```
/*  
    - GL_SMOOTH 默认的平滑模式  
    - GL_FLAT 平面模式  
*/  
void glColor(GLenum mode);
```

三维绘制和交互



三维绘制

在屏幕上产生3D效果可能用到了**透视**、**隐藏面的消除**、**颜色和着色**、**光照阴影**等

3D几何用**基本几何图元**构建

- 简单物体：编程构建
- 复杂物体：通过**建模软件构建**保存成**模型文件**，Ply，Obj等模型格式

高级几何对象

GLUT中的高级几何对象有

- 立方体
- 球体
- 圆锥
- 圆环
- 规则多面体
- 犹他壶

每种类型的对象均有**线框绘制**和**实体绘制**两种绘制函数

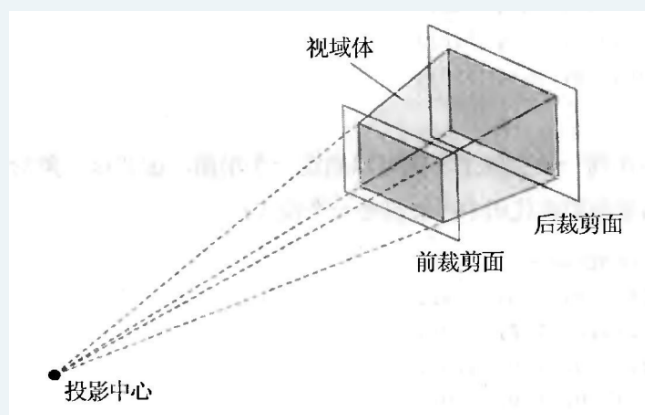
```
//Type指代绘制的对象类型  
glutWireType() // 线框  
glutSolidType() // 实体
```

合成摄像机模型

要让OpenGL变换能够正确的模拟真实摄像机的成像过程，需要提供正确的变换矩阵

- 投影矩阵：描述相机内部参数
- 模型视图矩阵：描述物体与相机之间的相对位置、方位关系

OpenGL模拟投影过程



- OpenGL通过**建立视域体**确定有哪些对象可见
- OpenGL通过**正交投影**、**透视投影**确定对象在投影平面上的位置
- 确定**对象的位置和朝向**
- 确定**摄像机的位置和朝向**、**透镜参数**(长焦、广角)、**底片尺寸**

摄像机的定位和定向

- 相对于**被观察对象**调整摄像机

```
void gluLookAt (
/*
    第一组是相机在世界坐标的位置
    第二组是相机镜头对准的物体在世界坐标的位置
    第三组相机向上的方向在世界坐标中的方向
*/
GLdouble eyex, GLdouble eyey, GLdouble eyez,
GLdouble atx, GLdouble aty, GLdouble atz,
GLdouble upx, GLdouble upy, GLdouble upz)
```

- 相对于**摄像机**调整被观察对象

```
// 平移
void glTranslate<fd>(type dx, type dy, type dz)
// 旋转
void glRotate<fd>(type angle, type dx, type dy, type dz)
// 缩放
void glScale<fd>(type sx, type sy, type sz)
```

投影取景

OpenGL投影取景的实现要设置**视域体**，视域体对三维场景进行裁剪，只有在视域体内的对象才能被观察

- **正交投影**

物距无穷大，可以理解为投影中心在无穷远处

模拟**长焦镜头**，可以保持距离和形状，但没有透视效果

```
// right > left , top > bottom, far > near
void glOrtho(GLdouble left, GLdouble right,
```

```
GLdouble bottom, GLdouble top,  
GLdouble near, GLdouble far)
```

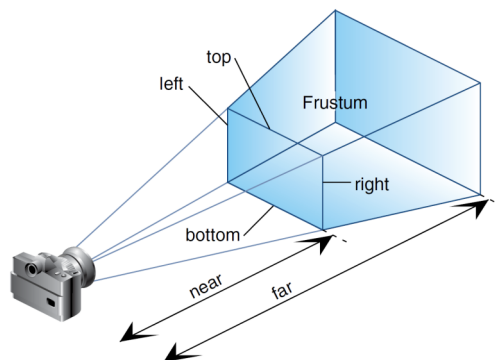
- **透视投影**

模拟各种**透视镜头**，有立体感，但会导致近大远小

投影平面可以设置在投影中心与前裁剪面之间的任何位置。

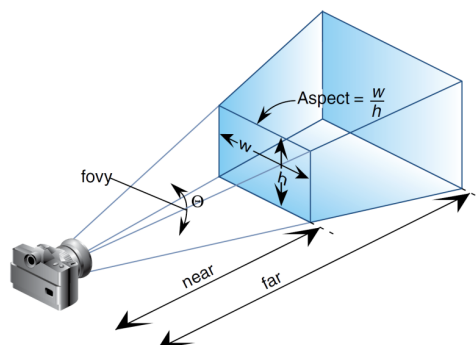
投影中心通常也称为**视点**

```
void glFrustum(GLdouble left, GLdouble right,  
GLdouble bottom, GLdouble top,  
GLdouble near, GLdouble far)
```



或用

```
void gluPerspective(GLdouble fovy, GLdouble Aspect,  
GLdouble near, GLdouble far)
```



交互

回调函数

隐藏面消除

隐藏面消除是指消除多边形图元的不可见表面，可以通过

- 多边形拣选

```
glEnable(GL_CULL_FACE)
glCullFace(GL_BACK) // 反面不可见
```

可见的几何图元是**最后绘制的几何图元**

- 深度缓存算法(Zbuffer)

增加**额外的深度缓存空间**来保存在绘制过程中绘制器**已经绘制图元的深度信息**

在绘制新图元时**比较深度值**，判断是否遮挡

```
// 在main()中请求一个深度缓存
glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE | GLUT_DEPTH)
// 在init()中OpenGL的初始化阶段，启用深度缓存功能
glEnable(GL_DEPTH_TEST)
// 在display()中绘制开始前，清空颜色缓存的同时，通常也清空深度缓存
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

可见的几何图元是**离摄像机更近的图元**

鼠标回调函数

```

/*
- button: 哪个按钮导致事件发生
- GLUT_LEFT_BUTTON:左键
- GLUT_RIGHT_BUTTON:右键
- GLUT_MIDDLE_BUTTON:中键
- state: 按钮的状态
- GLUT_UP
- GLUT_DOWN
- x,y: 窗口中位置，左上角为原点
*/
glutMouseFunc(mouse)
void mouse(int button, int state, int x, int y)

```

由于y轴向下，获取位置需要知道窗口的尺寸，而窗口的尺寸是可能改变的
因此，要用一个**全局变量跟踪其变化**，**新高度值返回给形状改变回调函数**

键盘回调函数

当鼠标位于窗口内，并且键盘有某个键被按下或释放，就会产生键盘事件

```

// 普通按键
glutKeyboardFunc(keyboard)
void keyboard(unsigned char key,
int x, int y) // key是被按下键对应的ASCII码
// 特殊按键在glut.h中有定义
if(key == GLUT_KEY_F1) //F1
if(key == GLUT_KEY_UP) //向上
// 修饰键: ctrl,alt,shift
// ctrl+C退出程序
if (glutGetModifiers() == GLUT_ACTIVE_CTRL) &&(key == 'c' || key =
= 'C')

```

INFO.

GLUT中释放键不算一个事件

形状改变回调函数

拖动窗口会导致窗口尺寸改变，导致对象的形状改变，需要形状改变回调函数

```
/*  
    - 返回新窗口的宽度与高度（单位：像素）  
    - 回调函数执行后自动发送刷新显示事件，触发显示回调  
*/  
glutReshapeFunc(reshape)  
void reshape(int w, int h)
```

菜单

GLUT支持**弹出式菜单**(也可以有子菜单)

创建步骤

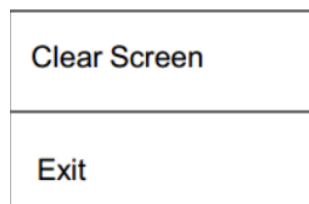
- 定义**菜单内各条目**
- 定义**每个菜单项的行为**
- 把菜单**连接到鼠标按钮上**

```
// 创建一个使用回调函数f()的菜单，并返回菜单的整数标识符  
int glutCreateMenu(void (*f)(int value))  
// 为当前菜单增加一个名为name的菜单项；value值在选中时返回给菜单回调函数  
void glutAddMenuEntry(char *name, int value)  
// 将当前菜单关联到鼠标按钮button(左右中键)上  
void glutAttachMenu(int button)
```

例如

```
// main函数中
menu_id = glutCreateMenu(mymenu);
glutAddMenuEntry("Clear Screen", 1);
glutAddMenuEntry("Exit", 2);
glutAttachMenu(GLUT_RIGHT_BUTTON);
// mymenu菜单回调函数
void mymenu(int value) {
    if(value==1)
        glClear(GL_COLOR_BUFFER_BIT);
    if(value==2) exit(0);
}
```

渲染结果如下

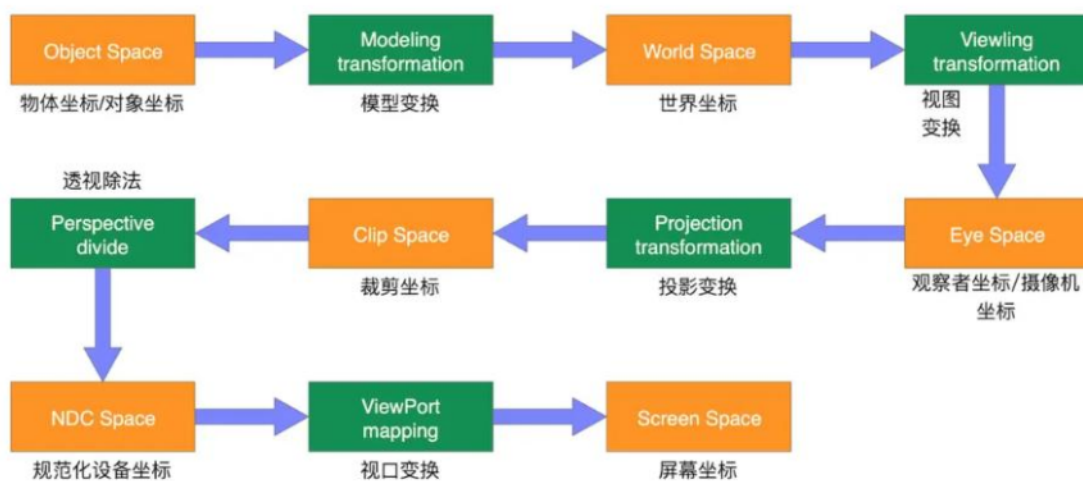


视图



顶点变换

顶点变换是物体坐标到屏幕坐标的过程，可由下图表示



模型视图变换

模型变换

下面的坐标为四元的，第四元为0时说明这代表一个**方向向量**，否则为一个**点**

- 平移

平移**改变物体的位置**，由平移向量 d 确定

设 $d = (d_x, d_y, d_z)$ ，有平移矩阵为

$$T = T(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

也记为 $T(d_x, d_y, d_z)$

- 旋转

先介绍二维旋转

二维旋转绕原点旋转 θ 度，等效于**三维空间绕z轴旋转**，从z轴正方向看为逆时针方向

有旋转矩阵为

$$R = R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

类似地，也可以得到绕x轴的旋转矩阵和绕y轴的旋转矩阵

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

旋转和平移均为**刚体变换**，刚体变换的复合不会改变对象的形状和体积

- **绕原点的一般旋转**

绕过原点任一轴旋转 θ 角可以分解为绕x, y, z轴旋转的复合，称 $\theta_x \theta_y \theta_z$ 为**欧拉角**

记旋转为

$$R(\theta) = R_z(\theta_z)R_y(\theta_y)R_x(\theta_x)$$

因为矩阵乘法不具有交换性，因此调换z, y, x的顺序将导致**不同的旋转效果**。

调换顺序之后，如果为了得到原来的旋转效果，则旋转角度应相应改变

- **不动点为 p_f 的旋转**

先将不动点移到原点，旋转后再移回去

$$M = T(p_f)R(\theta)T(-p_f)$$

- 绕任意轴的旋转

首先将对象的中心和旋转轴移动到原点，用两次旋转使得旋转轴r与z轴对齐，再绕z轴旋转

$$\mathbf{R} = \mathbf{R}_x(-\theta_x)\mathbf{R}_y(-\theta_y)\mathbf{R}_z(\theta)\mathbf{R}_y(\theta_y)\mathbf{R}_x(\theta_x)$$

对旋转轴r归一化处理，有 $r = (r_x, r_y, r_z)^T$

归一化处理是指处理为单位向量

则有

$$\mathbf{R} = \begin{pmatrix} \cos \phi + (1 - \cos \phi)r_x^2 & (1 - \cos \phi)r_x r_y - r_z \sin \phi & (1 - \cos \phi)r_x r_z + r_y \sin \phi \\ (1 - \cos \phi)r_x r_y + r_z \sin \phi & \cos \phi + (1 - \cos \phi)r_y^2 & (1 - \cos \phi)r_y r_z - r_x \sin \phi \\ (1 - \cos \phi)r_x r_z - r_y \sin \phi & (1 - \cos \phi)r_y r_z + r_x \sin \phi & \cos \phi + (1 - \cos \phi)r_z^2 \end{pmatrix}$$

对齐次坐标，有

$$M = T(p_0)RT(-p_0)$$

- 缩放

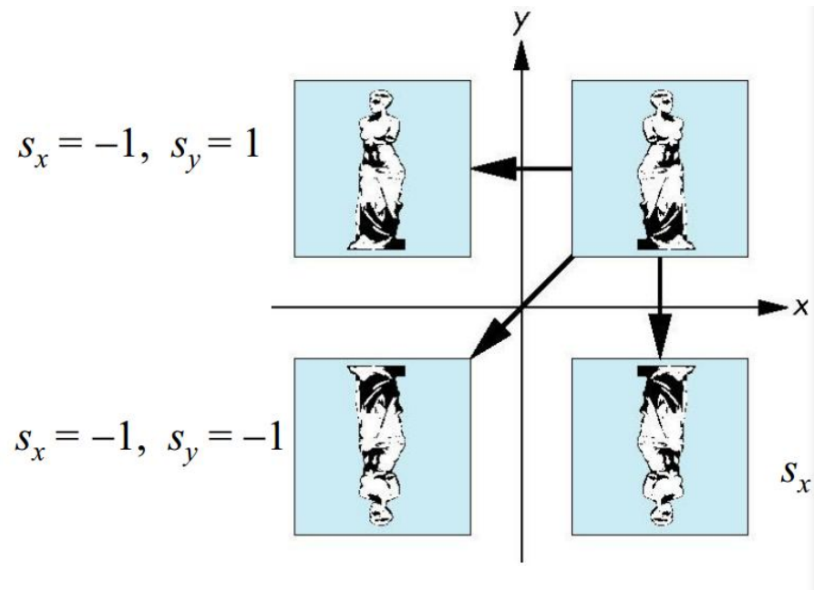
缩放使得对象沿每个坐标轴伸展或收缩，设缩放向量为 $s = (s_x, s_y, s_z)$

有缩放矩阵

$$S = S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 反射

反射是一种**特殊的缩放**，**缩放向量出现了负数**，使得对象向反方向伸缩



相机变换

也称为**视点变换**，相机变换包括**三个要素**

- eye position
- gaze direction
- view-up vector

分别简记为e,g,t, 则有

$$w = -\frac{g}{||g||}$$

$$u = \frac{t \times w}{||t \times w||}$$

$$v = w \times u$$

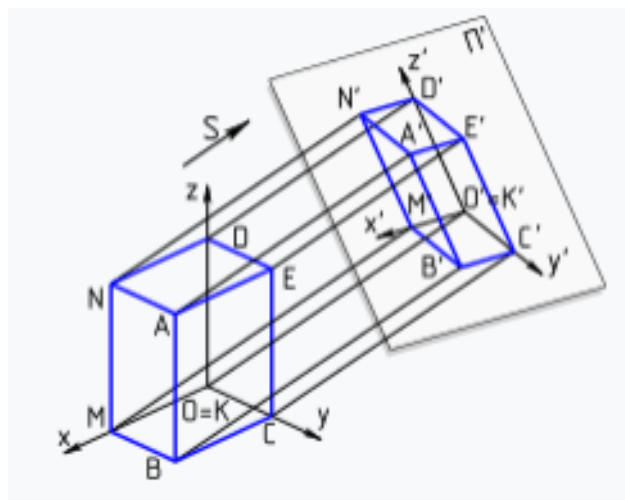
得到**相机变换矩阵**为

$$M_{cam} = \begin{bmatrix} u & v & w & e \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} x_u & y_u & z_u & 0 \\ x_v & y_v & z_v & 0 \\ x_w & y_w & z_w & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_e \\ 0 & 1 & 0 & -y_e \\ 0 & 0 & 1 & -z_e \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

投影变换

投影包括**透视投影**、**平行投影**、**正交投影**

平行投影



投影线之间**相互平行**

正交投影

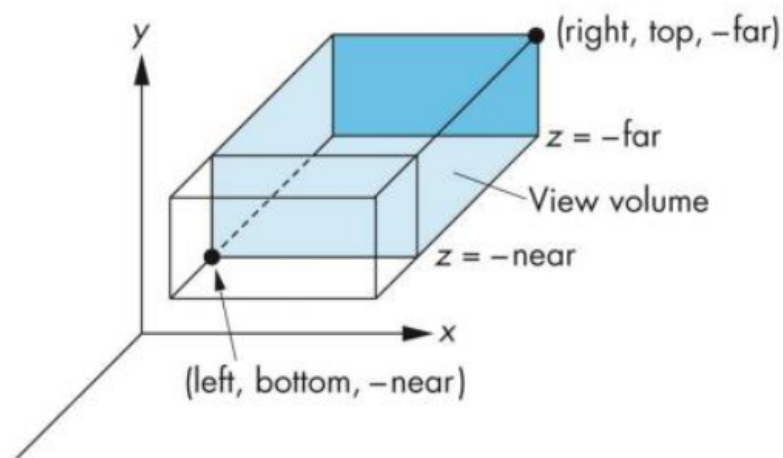
正交投影是一种**特殊的平行投影**，要求**投影线垂直于平面**

正交投影保持了距离、角度和形状，但看不到对象真正的全局形状，多用于工业零件等

在glut中，可以这样使用正交投影

```
// 视景物是与坐标轴平行的长方体
// near, far可取正值、零和负值，但near和far不应相同
glMatrixMode(GL_PROJECTION);
```

```
void glOrtho(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top, GLdouble near, GLdouble far);
```



i 规范视景体

OpenGL缺省的视景体是中心在原点，边长为2的立方体，相当于调用

```
glMatrixMode(GL_PROJECTION);  
glLoadIdentity();  
glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
```

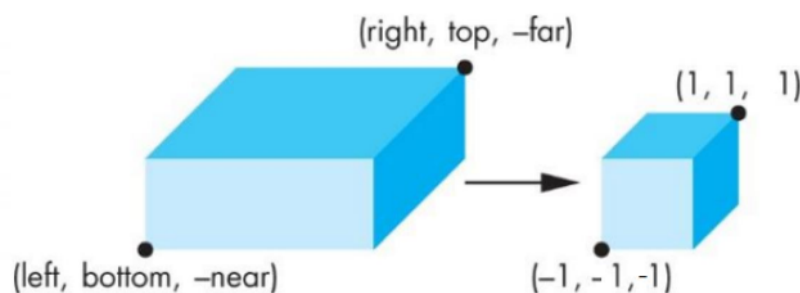
称这个视景体为**规范视景体**

采用规范视景体可以在**同一流水线**支持**平行投影**和**透视投影**，同时又简化了裁剪过程

正交规范化

正交规范化指的是求出把**指定裁剪体**转化为**默认裁剪体**的变换

```
// 把视景体内的坐标规范化到-1和1之间  
glOrtho(left, right, bottom, top, near, far)
```



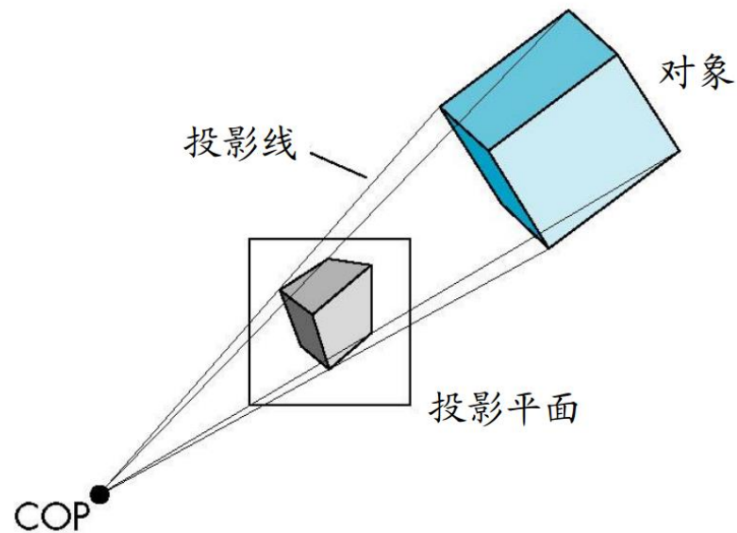
近裁剪面 $z=-near$ 映射到 $z=-1$ 平面，
远裁剪面 $z=-far$ 映射到 $z=1$ 平面

正交规范化的变换首先**将中心移动到原点**，再进行**缩放**使得**视景体边长为2**

可得**正交规范化变换矩阵**为

$$M_{orth} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

透视投影

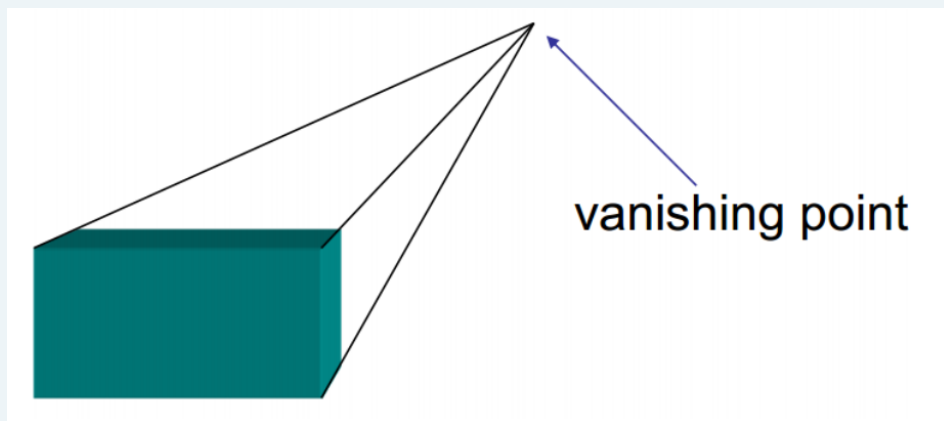


投影线会聚于投影中心(COP)，尺寸缩小

观察者相对于投影平面对称，投影中心和投影窗口确定一个对称的正棱锥

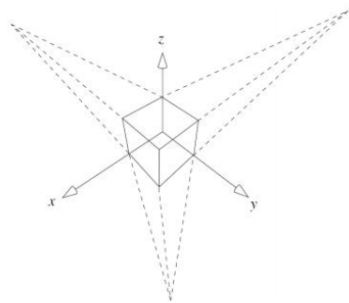
i 灭点

灭点是对象上（不平行于投影面）的平行线在投影后的相交点



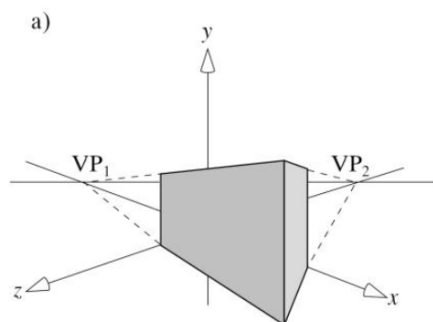
根据灭点的多少，相应的有

- 三点透视



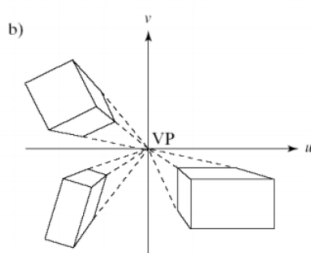
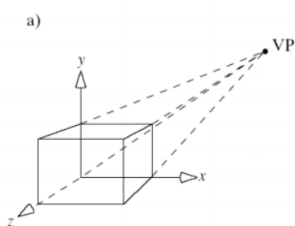
(a)

• 两点透视



(b)

• 单点透视



同样大小的对象，离视点越远，投影结果就越小，更符合人的视觉习惯

但直线上等距的几点投影后不一定等距，也即出现了**非均匀缩短**，因此难以手工绘制，多用于动画

在glut中实现透视变换，可以用 `glFrustum` 定义视景体



TIP.

gluPerspective也可以，但是只能定义对称视景物

透视变换的投影矩阵为

$$M_{per} = M_{orth}P$$

其中，P为

$$P = \begin{bmatrix} -n & 0 & 0 & 0 \\ 0 & -n & 0 & 0 \\ 0 & 0 & -(n+f) & -nf \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

也即

$$M_{per} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

视口变换

视口变换将坐标转换为屏幕坐标，也即把规范化视景物中的内容填入指定的范围内

可用

```
// 0x,0y为原点坐标，width和height为窗口宽高  
glViewport(0x, 0y, width, height)
```

视口变换矩阵为

$$M_{viewport} = \begin{bmatrix} \frac{w}{2} & 0 & 0 & \frac{w}{2} \\ 0 & \frac{h}{2} & 0 & \frac{h}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

z方向的信息交给**深度缓冲处理**

有

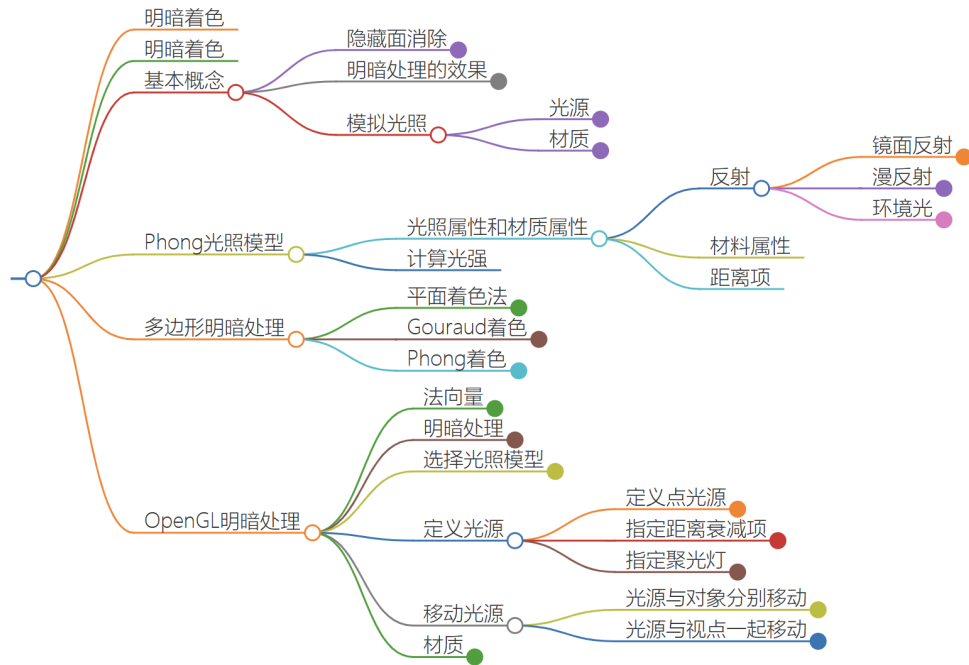
$$\begin{cases} WinX = Ox + \frac{w}{2}(x + 1) \\ WinY = Oy + \frac{h}{2}(y + 1) \\ WinZ = \frac{z + 1}{2} \end{cases}$$



TIP.

这里的WinY坐标和窗口系统的坐标（例如鼠标获取）朝向相反

明暗着色



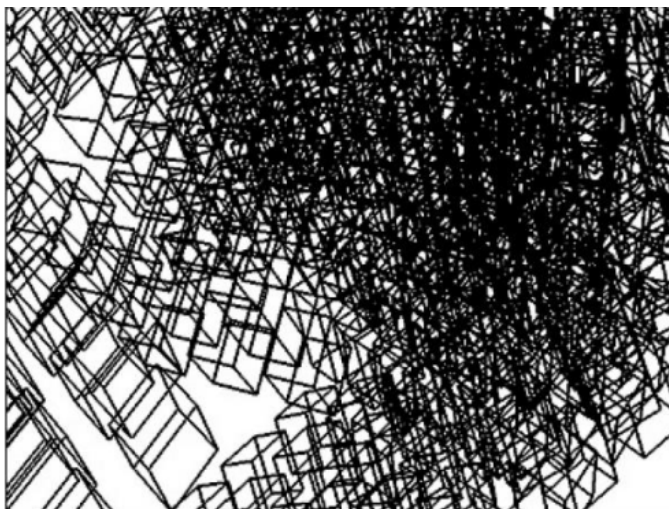
明暗着色

明暗着色介绍了光源作用在物体上呈现出不同明暗效果的物理过程

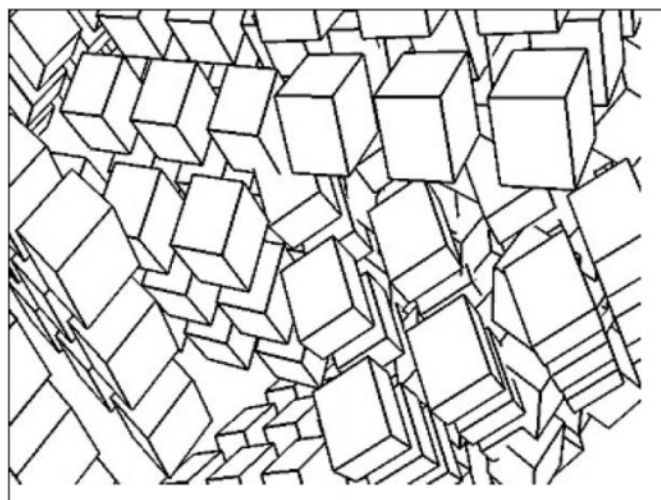
基本概念

隐藏面消除

在计算机图形学中的模型通常由多个多边形网格构成



为了使得多边形网格显得真实，表现遮挡关系，可以除去那些被遮挡的面，那么有



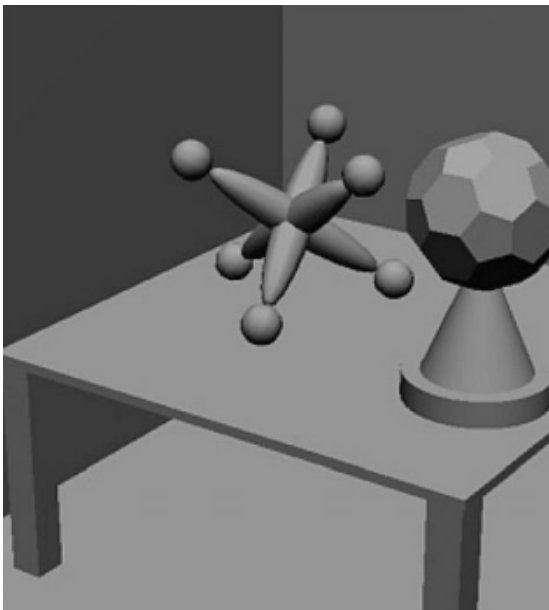
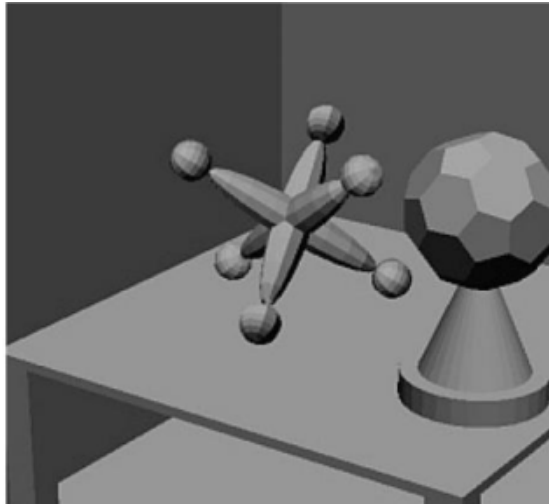
明暗处理的效果

仅仅消除了隐藏面，观感仍显得立体，为此要模拟光照在物体的状态

因此需要充分根据**对象表面的状态、光源的位置、视点的位置**确定每个像素的**颜色亮度**

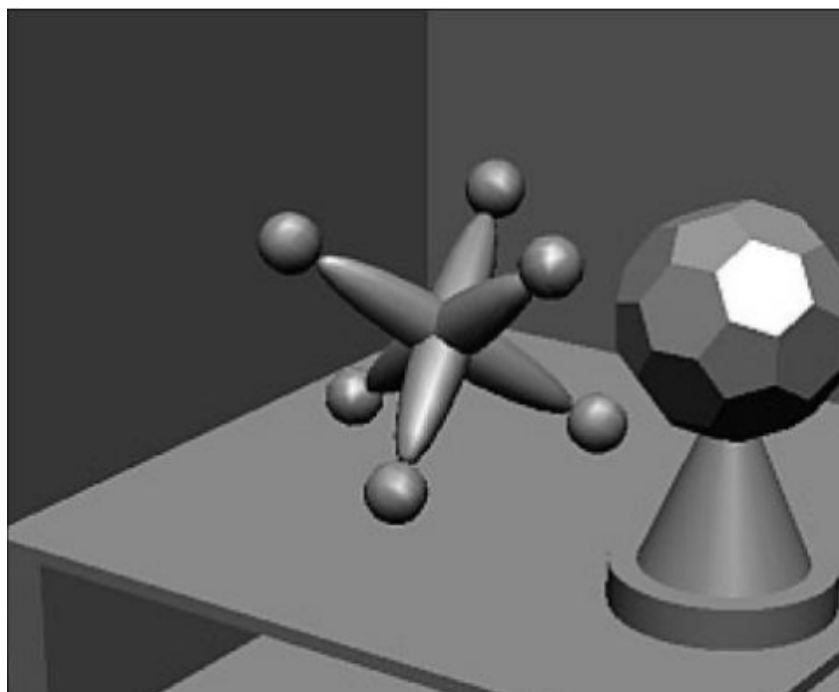
- **光滑明暗处理**

若多边形网格表示光滑对象，那么显示的结果应该反映这种光滑性，在计算每个顶点处亮度后以**线性插值**计算亮度

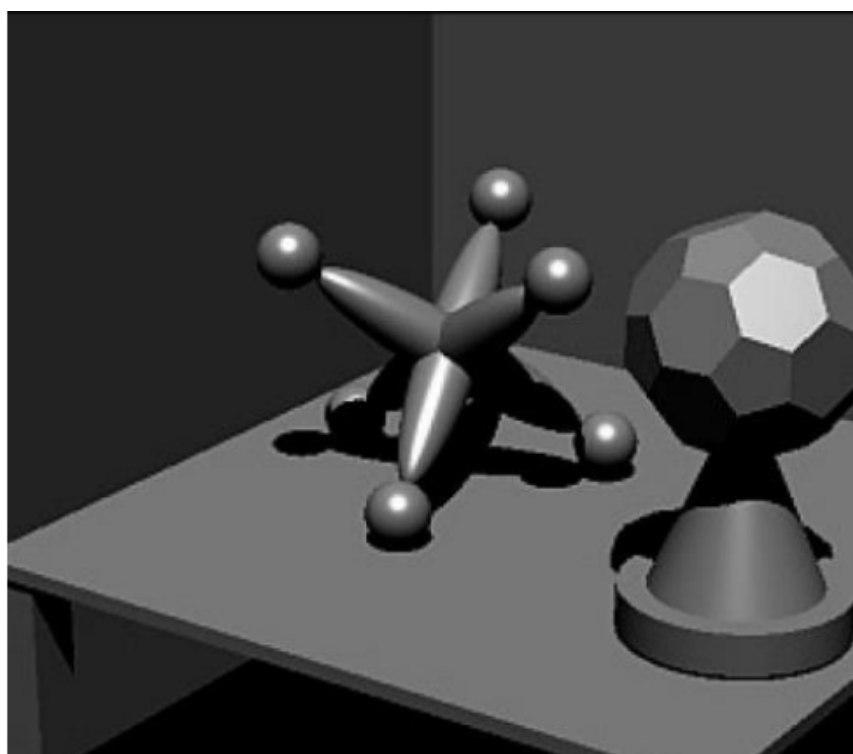


明显，第二张图经过了明暗处理，这个算法也称为**Gouraud明暗处理算法**

- 镜面光



- 阴影



- 纹理映射



模拟光照

模拟光照可以通过**光线跟踪**、**辐射度方法**实现

- **光线跟踪**

容易实现，模拟真实的光照反射、散射，但**计算复杂**

- **辐射度方法**

基于**能量守恒**

i 局部与全局光照

正确的明暗处理需要全局计算，即包含了所有的对象和光源，但这与流水线模型不兼容

为此，要对每个多边形**单独**进行**明暗处理(局部光照)**

局部光照也可以通过**多种手段逼近全局效果**

实现模拟光照，需要考虑**光源**、**材质**、**观察者位置**、**曲面定向**

光源

光线从光源表面离开的方式有**自发射**和**反射**两种

在表面上任一点 (x, y, z) 所发出的光可以用**发射方向** (θ, φ) 和波长 λ 的光强描述

那么可以通过**照明函数** $I(x, y, z, \theta, \varphi, \lambda)$ 确定一个光源



TIP.

一般的光源难以处理，因为要对在光源表面上的所有点的光强进行积分

大多数应用下，我们用**光的三种成分的强度**表示光的亮度，有**光亮度函数**

$$L = [L_r, L_g, L_b]$$

点光源

点光源由**位置**和**颜色**表示，理想的点光源向**各个方向发射光线**的强度相等

设接收光的点为 p ，光源位于 p_0 ，有

$$L(p, p_0) = \frac{1}{|p - p_0|^2} L(p_0)$$

其中， $L(p_0) = [L_r(p_0), L_g(p_0), L_b(p_0)]$ 为**点光源的亮度函数**

- **点光源**在计算机图形学中大量应用，因为它易于使用
- 但点光源无法很好地反映物理现实，只有点光源的场景得到的图像中**对比度较高**，对象显得要么很亮，要么很暗。

真实的光源由于尺寸大，因此场景结果较为柔和

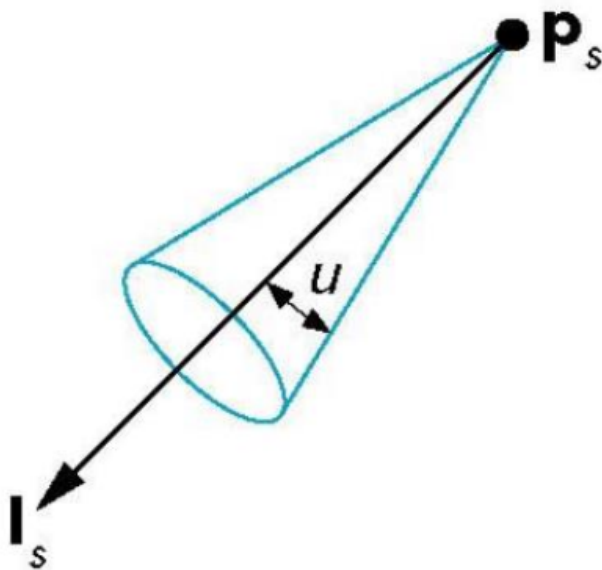
- 可以在点光源处**加入环境光**降低高对比度的问题



本影和半影

本影是**完全在阴影中的区域**，半影是**部分在阴影中的区域**

聚光灯



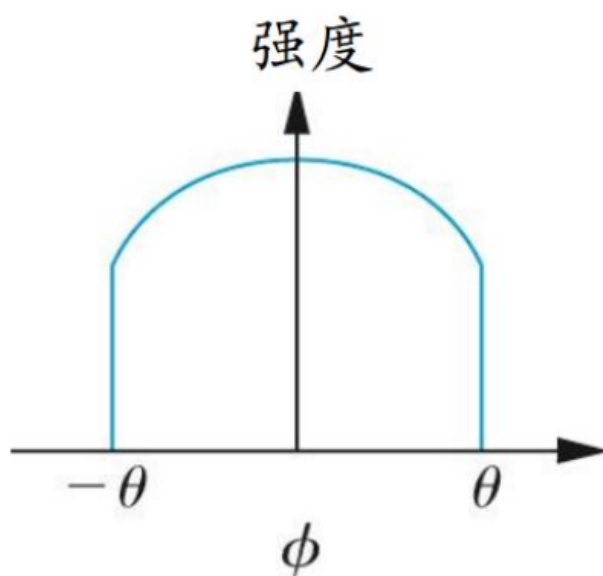
聚光灯具有一个**比较窄的照明范围**，通常为**圆锥形半无穷区域**



TIP.

实际上，对点光源做一定的限制就可以得到聚光灯，当中心轴和母线夹角为 180° 时，聚光灯为**点光源**

聚光灯的**光亮度分布**如下图



强度函数一般为

$$L = \cos^e \phi$$

其中， e 确定了**光强度衰减的快慢**

无穷远光源

无穷远光源的例子有太阳，在计算这样的光照时需要**照射表面的点指向光源的方向向量**

对于一个无穷远光源，向量会成为一个常数

环境光

环境光**模拟均匀光照**，场景中的每个点都**具有相同的亮度**

环境光亮度由 $L_a = [L_{ar}, L_{ag}, L_{ab}]$ 确定，每点值均相同，但**各个表面对环境光反射不同**

材质

光线在**对象表面部分被吸收，部分被散射或折射**

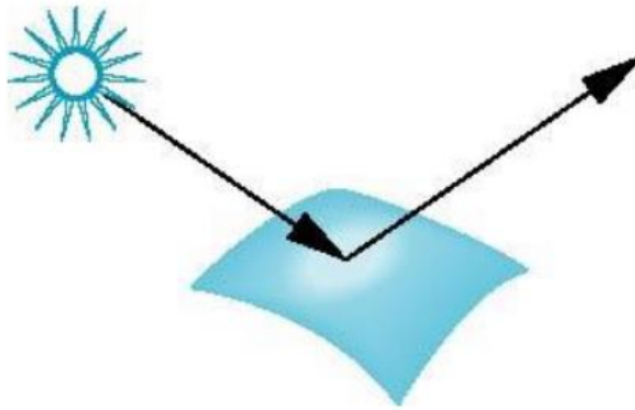
反射部分的多少确定**对象的颜色和亮度**，而反射光被反射的方式是由**表面的光滑程度和定向**确定的

光滑的表面对应一个明亮的对象，否则对象会变得暗淡

- **镜面**

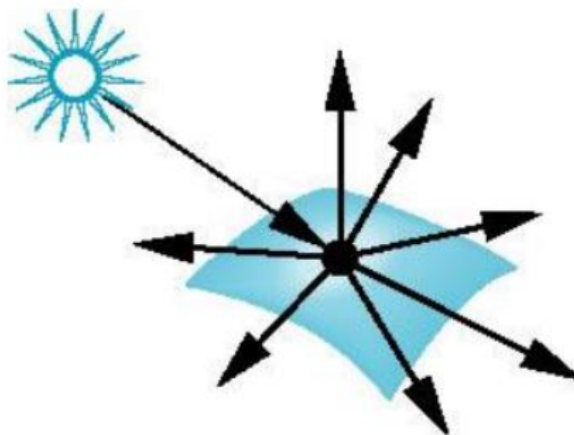
绝大多数光集中在**严格镜面反射方向的周围，表面明亮**

入射光除**部分被吸收**外，全部以**单一角度反射**



- **漫反射面**

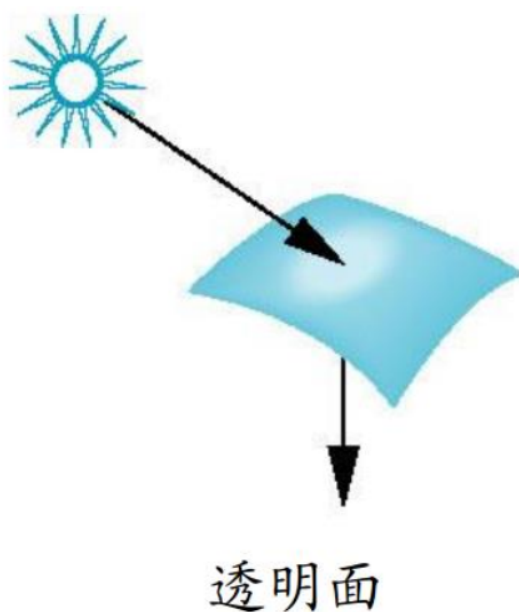
漫反射面的反射光线**向各个方向散射**，理想漫反射面把光线**均匀散射到各个方向**



漫反射面

- **透明面**

对于透明面，要**考虑折射光**



在三维场景中的每个对象的表面可以是上述三种情形中的任一种，也可以是其中两种或三种的综合

每种情形所占比例由对象表面的性态确定

Phong光照模型

Phong光照模型是可以快速计算的局部光照模型

考虑漫反射光、镜面反射光、环境光，采用入射光方向 l 、视点方向 v 、法向 n 、理想反射方向 r

在Phong光照模型下，我们用局部光照模型模拟全局光照效果

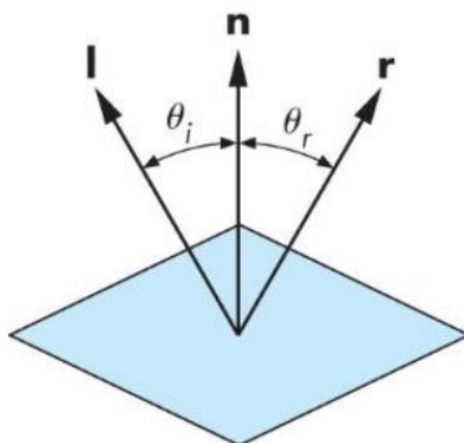
光照属性和材质属性

假设点光源的三原色都有各自的环境光、漫反射光和镜面反射光，那么就需要9个参数描述光源在 p 处的光照属性(不考虑距离衰减)

光照模型将每个光源的结果叠加在一起

反射

光照模型的前提是已知某点的入射光，通过某种方法求得在该点的反射光



由反射定律，可得

$$\mathbf{r} = 2(\mathbf{l} \cdot \mathbf{n})\mathbf{n} - \mathbf{l}$$

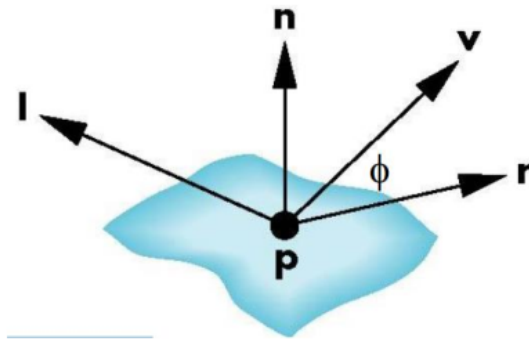
这里的三个向量均取单位向量(便于求夹角)

镜面反射

光滑表面之所以显出镜面高光，是因为入射光反射后，绝大多数集中在理想反射方向周围

镜面光模型考虑

- n : 法向量
- l : 入射光方向
- r : 理想反射方向
- v : 视点方向
- ϕ : r 与 v 的夹角, $\cos \phi = \mathbf{v} \cdot \mathbf{r}$



反射光强满足下面的关系

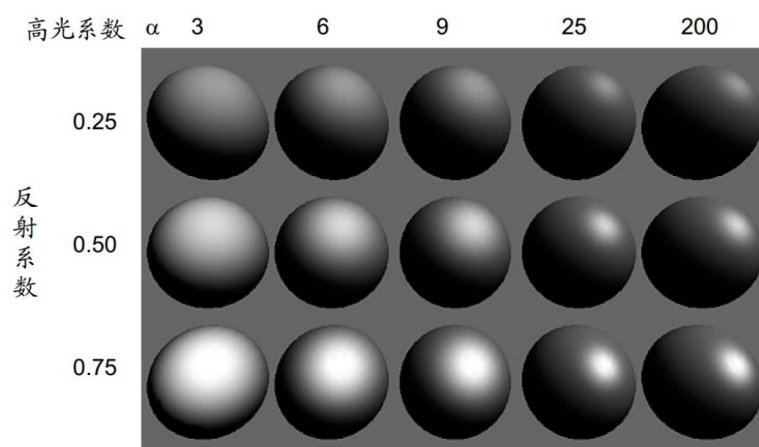
$$I_s \propto k_s L_s \cos^\alpha \phi$$

其中, I_s, k_s, L_s, α 分别为**反射光强**, **反射系数**, **入射光强**, **高光系数**

高光系数与材质的关系

- a的值在**100~200**间, 对应于**金属材料**
- a的值在**5~10**间, 对应于**塑料**
- a的值**无穷大**时, 对应于**镜子**

下面的图反映了镜面反射参数的影响



漫反射

用**郎伯表面**来模拟粗糙表面, 光向**各个方向均匀地**散射

反射光的比例**正比于入射光的竖直分量**，即 $I \propto \cos \theta_i$

存在三个系数 k_{dr}, k_{dg}, k_{db} 分别相应于每种颜色的**光反射的比例**

对每种光，**漫反射光强为**

$$I_d = k_d(\mathbf{l} \cdot \mathbf{n})L_d$$

其中， $(\mathbf{l} \cdot \mathbf{n}) = \max(\mathbf{l} \cdot \mathbf{n}, 0)$

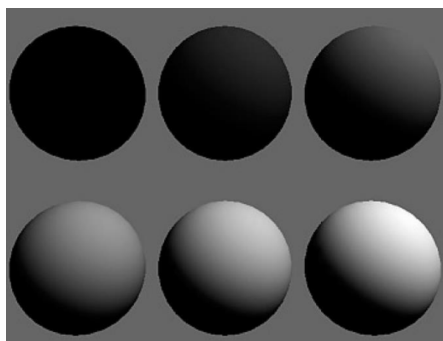
INFO.

基于简单Lambertian漫反射模型所生成的对象图形**显得比较暗淡**

因为此时相当于假定在视点处有一个点光源，那些光源无法直接照到的地方显得较黑

在真实的场景中，对象也接受到其它对象反射过来的光

逐渐增大漫反射系数，保持光强不变



环境光

环境光是由于在场景中（大的）光源**与对象间的多次相互作用**而导致的，其量与颜色**依赖于光源的颜色和对象的材料属性**

环境光光强满足

$$I_a = k_a L_a$$

其中， k_a, L_a 分别为反射系数，环境光强

向漫反射光中加入环境光，逐渐提高反射系数



材料属性

与光源属性对应，对三原色的三种反射情况有**不同的反射系数**，此外还有一个**高光系数** α ，因此共计10个系数

距离项

距离项**模拟光的衰减**，可以在漫反射项和镜面项添加二次距离衰减因子 $\frac{1}{(a+bd+cd^2)}$ ，其中d表示距离

常数与线性项起到**柔和点光源**的效果

计算光强

对于每个光源和每种颜色成分，Phong光照模型可以表示为

$$I = k_d L_d \max(\mathbf{l} \cdot \mathbf{n}, 0) + k_s L_s \max((\mathbf{v} \cdot \mathbf{r})^\alpha, 0) + k_a L_a$$

对每个颜色分量，把所有**光源贡献的值加在一起**

中值向量

在Phong模型中，镜面光项要为每个顶点计算一个新的反射向量和视点向量，为了提高效率，用中值向量给出一个近似

中值向量h表示为

$$h = \frac{\mathbf{l} + \mathbf{v}}{|\mathbf{l} + \mathbf{v}|}$$

n 和 h 的夹角 ψ 称为**中值角**，当 v 位于 l 、 n 和 r 所在平面时，可以证明
 $2\psi = \phi$

对于镜面项，可以用 $(n \cdot h)^\beta$ 代替 $(v \cdot r)^\alpha$

由此得到的模型称为改进的Phong模型或者**Blinn光照模型**

多边形明暗处理

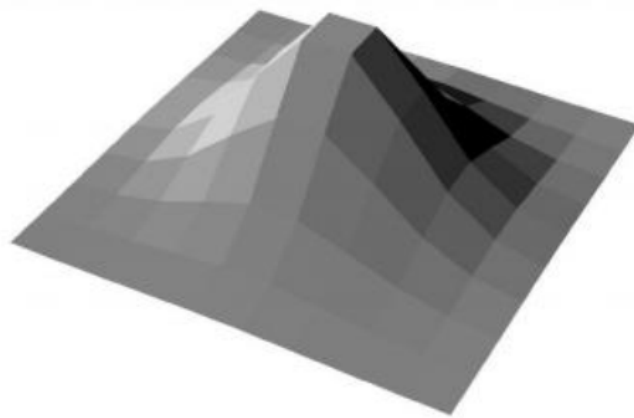
对于一个用多边形逼近的曲面，多边形明暗处理是指确定各个**多边形内部像素的颜色**

问题可转化为：已知**多边形四个顶点的位置、颜色(光照)、法向**，求多边形内各个像素的颜色

平面着色法

在同一多边形中**法向量 n 为常向量**，令视点和光源均为无穷远，则**视点方向 v 和入射方向 l 为常向量**

从而对于每个多边形，用其上一点的颜色代替整个多边形的颜色



OpenGL中设置平面着色为

```
// - 单个多边形(GL_POLYGON)，第1个顶点  
// - 独立三角形(GL_TRIANGLES)，第3i个顶点
```

```
// - 独立四边形(GL_QUADS), 第4i个顶点  
// - 四边形带(GL_QUAD_STRIP), 第2i+2个顶点  
// - 三角形带或三角形扇, 第i+2个顶点  
glShadeModel(GL_FLAT);
```

平面着色使得**网格中每个多边形的颜色不同**

如果多边形网格表示的是一个光滑曲面，那么这种效果显然是不令人满意的

Gouraud着色

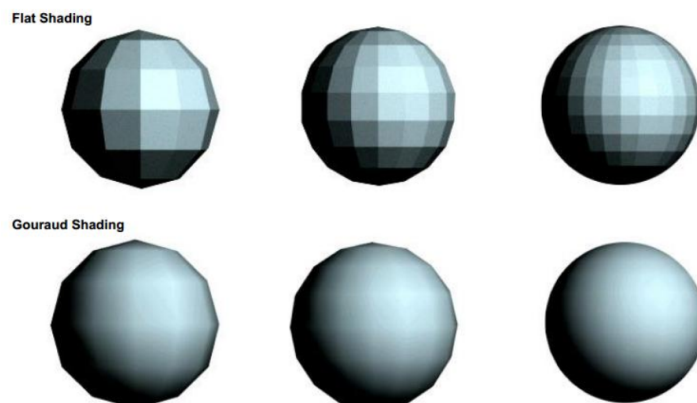
Gouraud着色对颜色进行**双线性插值**

1. 设定一个**水平扫描线**，**自上而下**
2. 对每条扫描线计算其与多边形的**左右交点位置及其颜色**
3. 对扫描线上的每一点**按线性插值得到颜色**

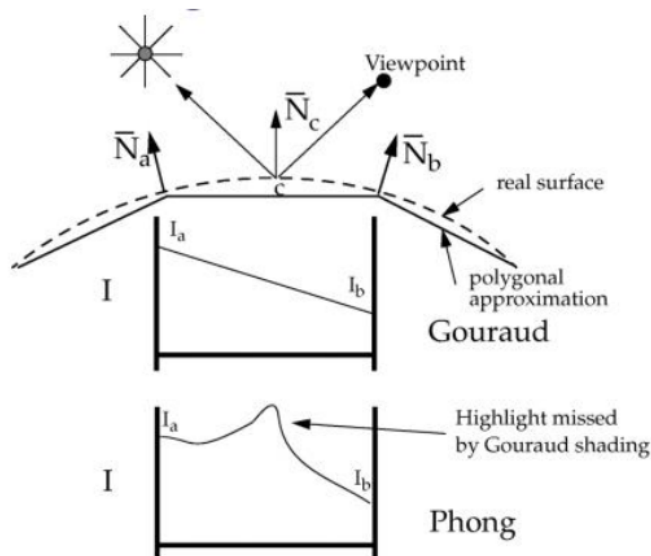
Opengl中设置**Gouraud着色**为

```
glShadeModel(GL_SMOOTH);
```

Gouraud着色下，同一个多边形上的颜色**出现了渐变**



但Gouraud着色会出现高光丢失的问题，假设a,b两点间应有一点高光c，线性插值会使得c的高光丢失



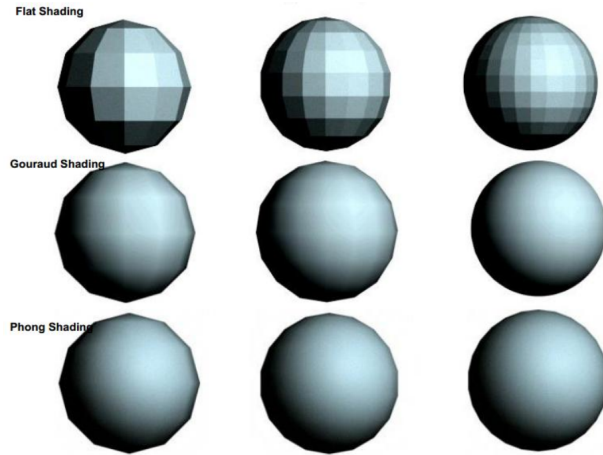
Phong着色

Phong着色对**法向插值**，用插值后的**法向**计算**光照**

1. 设定一个**水平扫描线**，**自上而下**
2. 对每条扫描线计算其与多边形的**左右交点位置**及其**法向**
3. 对扫描线上的每一点按**线性插值得到法向**
4. 用**光照模型**计算得到该点颜色

i 法向求平均

设有法向量 $\mathbf{n}_1, \mathbf{n}_2$ ，其平均为 $\bar{\mathbf{n}} = \frac{\mathbf{n}_1 + \mathbf{n}_2}{|\mathbf{n}_1 + \mathbf{n}_2|}$



OpenGL并没有内置Phong着色模型，因为法向计算较为复杂，很难得到**实时**图像

在多边形逼近大曲率曲面时，Phong的结果会比Gouraud明显光滑

OpenGL明暗处理

OpenGL中的明暗处理需要

1. 指定法向量
2. 启用明暗处理功能
3. 选择模式
4. 指定光源和材料属性

法向量

法向量在OpenGL中用

```
glNormal3d(x, y, z);  
glNormal3dv(p); // 顶点数组
```

设置

一般令法向量为单位向量，可以简化余弦运算，需要启用

```
// 二者选其一  
glEnable(GL_NORMALIZE);  
glEnable(GL_RESCALE_NORMAL);
```

自动进行法向量的单位化

明暗处理

明暗处理的启用需要

```
glEnable(GL_LIGHTING);
```

! WARN.

如果光照被激活， glColor()命令将被忽略
随后，要单独激活各个光源 i ， ($i=0,1,...,7$)

```
glEnable(GL_LIGHTi);
```

选择光照模型

光照模型参数包括

- 全局环境光强度
 - 观察点位于场景中还是无限远处
 - 物体的正反面是否执行不同的光照计算

- **镜面光颜色**是否从**环境光**和**漫反射**颜色中分离出来，并在**纹理操作**后再应用

```
void glLightModel{if}(GLenum pname, TYPE param);
void glLightModel{if}v(GLenum pname, TYPE *param);
```

- `GL_LIGHT_MODEL_AMBIENT, (0.2,0.2,0.2,1.0)` :

整个场景中的**全局环境光强**

- `GL_LIGHT_MODEL_LOCAL_VIEWER` :

0.0或`GL_FALSE`, 在计算中**应用无穷远视点**的假设简化计算

- `GL_LIGHT_MODEL_TWO_SIDE`

0.0或`GL_FALSE`, 单独对**多边形的两面进行明暗处理**

- `GL_LIGHT_MODEL_COLOR_CONTROL`

`GL_SINGLE_COLOR`(默认)或`GL_SEPARATE_SPECULAR_COLOR`, **镜面光**是否与**漫反射**和**环境光**分开计算 (主要与纹理颜色相关)



TIP.

环境光依赖于每个光源的颜色，因此需要对每个光源指定环境光强

定义光源

光源具有的参数为

- **光照类型及强度**
- **位置**
- **衰减项**
- **聚光灯参数**

```
void glLight{if}(GLenum light, GLenum pname, TYPE param);  
void glLight{if}v(GLenum light, GLenum pname, TYPE *param);
```

- **光照类型:**

- GL_AMBIENT
- GL_DIFFUSE
- GL_SPECULAR

- **位置:**

- GL_POSITION

- **衰减项:**

- GL_CONSTANT_ATTENUATION(常数项, 默认为1)
- GL_LINEAR_ATTENUATION(线性项, 默认为0)
- GL_QUADRATIC_ATTENUATION(二次项, 默认为0)

- **聚光灯参数:**

- GL_SPOT_CUTOFF
- GL_SPOT_DIRECTION
- GL_SPOT_EXPONENT

定义点光源

```
GLfloat diffuse0[]={1.0,0.0,0.0,1.0};  
GLfloat ambient0[]={1.0,0.0,0.0,1.0};  
GLfloat specular0[]={1.0,0.0,0.0,1.0};  
GLfloat light0_pos[]={1.0,2.0,3.0,1.0};  
glEnable(GL_LIGHTING);  
glEnable(GL_LIGHT0);  
glLightfv(GL_LIGHT0, GL_POSITION, light0_pos);  
glLightfv(GL_LIGHT0, GL_AMBIENT, ambient0);
```



```
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuse0);
glLightfv(GL_LIGHT0, GL_SPECULAR, specular0);
```

其中，光源颜色缺省值如下

- 环境光缺省值为(0,0,0,1)
- 漫反射光和镜面反射光对于光源0缺省值为(1,1,1,1)，其他为(0,0,0,1)

位置以齐次坐标形式给定

- w为1时，指定一个有限位置
- w为0时，指定一个平行光源，坐标给定入射光方向
- 其缺省值为(0,0,1,0)

指定距离衰减项

```
glLightf(GL_LIGHT0, GL_CONSTANT_ATTENUATION, 2.0);
glLightf(GL_LIGHT0, GL_LINEAR_ATTENUATION, 1.0);
glLightf(GL_LIGHT0, GL_QUADRATIC_ATTENUATION, 0.0);
```

指定聚光灯

```
glLightfv(GL_SPOT_DIRECTION, (0, 0, -1)); // 方向
glLightfv(GL_SPOT_CUTOFF, 180); // 角度范围
glLightfv(GL_SPOT_EXPONENT, 0); // 衰减指数
```

移动光源

光源是几何对象，它的位置或方向受模型视图矩阵的影响，其位置存储在视点坐标系中

光源与对象分别移动

需要依靠矩阵堆栈实现

```
static GLdouble spin;
void display(void){
    GLfloat light_pos[]={0.0,0.0,1.5,1.0};
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
    glPushMatrix();
    gluLookAt(0.0,0.0,5.0,0.0,0.0,0.0,0.0,1.0,0.0);
    glPushMatrix();
    glRotated(spin,1.0,0.0,0.0);
    glLightfv(GL_LIGHT0,GL_POSITION,light_pos);
    glPopMatrix();
    glutSolidTorus(0.275,0.85,8,15);
    glPopMatrix();
    glFlush();
}
```

光源与视点一起移动

要在视图变换之前设置光源位置

```
GLfloat ligh_pos[]={0.0,0.0,0.0,1.0};
glViewport(0,0,(GLsizei)w,(GLsizei)h);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluPerspective(40.0,(GLfloat)w/h,1.0,100.0);
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glLightfv(GL_LIGHT0,GL_POSITION,light_pos);
static GLdouble ex,ey,ez,upx,upy,upz;
void display(void){
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
```

```
glPushMatrix();
gluLookAt(ex,ey,ez,0.0,0.0,0.0,upx,upy,upz);
glutSolidTorus(0.275,0.85,8,15);
glPopMatrix();
glFlush();
}
```

材质

```
void glMaterial{if}(GLenum face, GLenum pname, TYPE param);
void glMaterial{if}v(GLenum face, GLenum pname, TYPE *param);
```

- face: GL_FRONT, GL_BACK, GL_FRONT_AND_BACK
- GL_AMBIENT, 缺省为(0.2, 0.2, 0.2, 1.0), **环境光**
- GL_DIFFUSE, 缺省为(0.8, 0.8, 0.8, 1.0), **漫反射光**
- GL_SPECULAR, 缺省为(0.0, 0.0, 0.0, 1.0), **镜面反射光**
- GL_SHININESS, 缺省为0.0, [0.0, 128.0], **高光指数**
- GL_EMISSION, 缺省为(0.0, 0.0, 0.0, 1.0), 表示材料本身的**发射光(类似光源)**, 不受其他任何光源或者变换的影响

颜色材料模式

通常启用光照后, glColor()会失去作用

如果使用了**glEnable(GL_COLOR_MATERIAL)**, 则光照模型中的几种光根据glColor()中的指定确定颜色

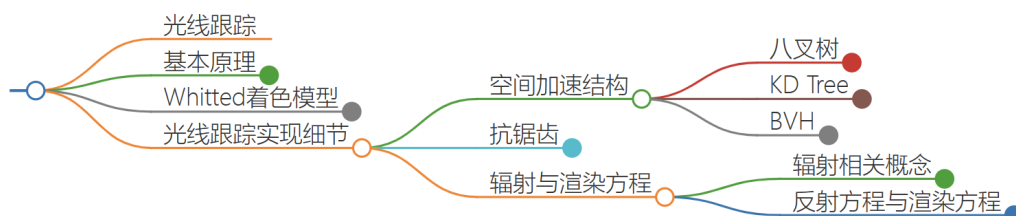
```
glEnable(GL_COLOR_MATERIAL);
glColorMaterial(GL_FRONT, GL_DIFFUSE);
/* now glColor* changes diffuse reflection */
glColor3f(0.2, 0.5, 0.8);
```

```

/* draw some objects here */
glColorMaterial(GL_FRONT, GL_SPECULAR);
/* glColor* no longer changes
diffuse reflection */
/* now glColor* changes specular >reflection */
glColor3f(0.9, 0.0, 0.2);
/* draw other objects here */
glDisable(GL_COLOR_MATERIAL);

```

光线跟踪



基本原理

光线跟踪和流水线方法同样是绘制的方法，但有一定的区别

1. 图像中的**每个像素将光线投射到场景中**，该光线的方向是通过追踪**从眼睛(相机)到像素**获得的。
2. 一旦确定了射线的方向，就开始**检查场景中的每个对象**，看射线是否与其**中的任何一个相交**。
3. 当发生**射线与多个对象相交的情况时**，选择**交点离眼睛(相机)最近的物体**。
4. 从**相交点向光源投射阴影射线**，如果这条特定的光线在通往光源的路上不**与某个物体相交**，那么**这个点就被照亮了**。

Whitted着色模型

Whitted着色模型将光强表示为

$$I_{\lambda} = L_{a\lambda}k_a + \sum_{lights} f_{att}L_{p\lambda}[k_d(\mathbf{n} \cdot \mathbf{l} + k_s(\mathbf{r} \cdot \mathbf{v})^n)] + k_r I_{r\lambda} + k_t I_{t\lambda}$$

其中, $I_{r\lambda}, I_{t\lambda}$ 分别为递归反射项和递归折射项

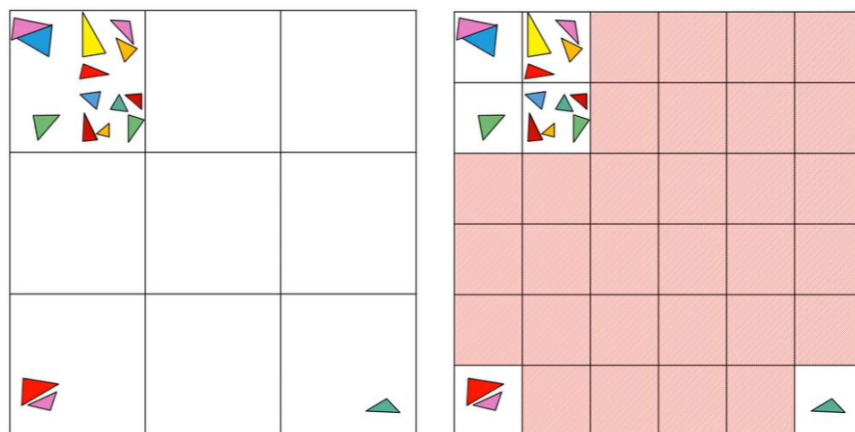
- 对每个点, 计算局部光照
- 如果是镜面或者折射面, 则继续递归计算
- 如果不是镜面或者折射面, 则仅包含局部光照

Whitted-Style方法是一种**不准确的模型**, 没有反映出完整的全局光照效果, 尤其是当物体表面不是镜面或者折射面时

光线跟踪实现细节

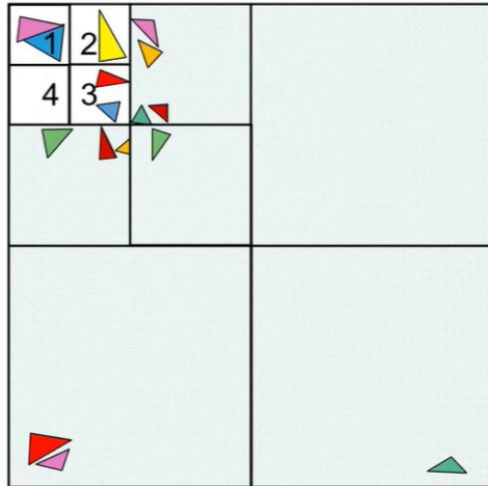
空间加速结构

空间加速结构通常指的是一种数据结构, 用于加速空间查询和检索操作



在上图中, 如果均匀分割网格, 则在物体分布不均的情况下, 需要更小的网格, 导致网格分割过多, 浪费空间和计算时间

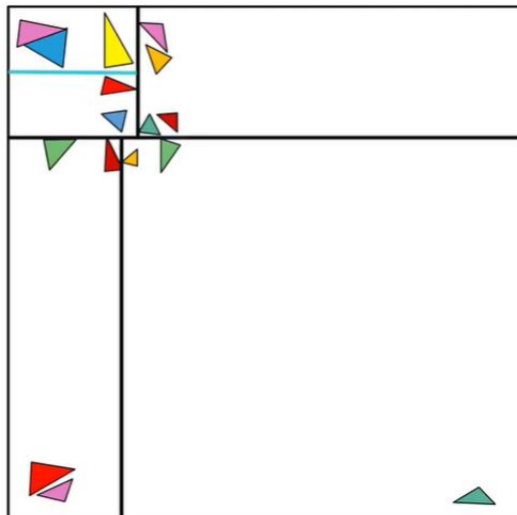
八叉树



当子空间为空或者三角形个数较少时，不再细分

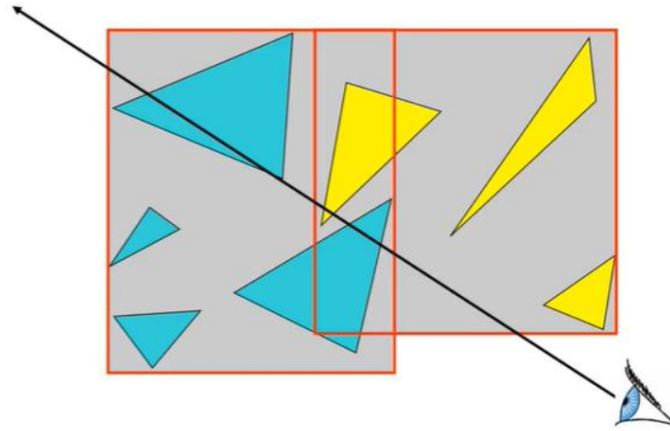
每次细分时原网格一分为八，深度加深时会带来指数爆炸

KD Tree



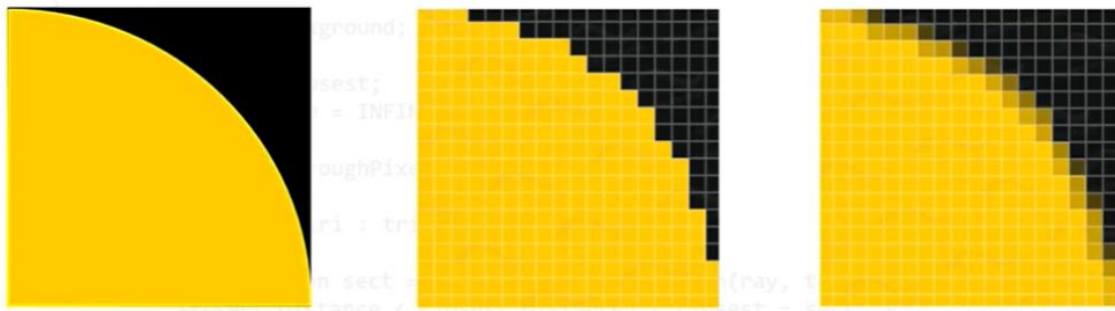
每次只分一个维度，但是会导致某个图元隶属于左右两个空间

BVH



类似KD Tree的划分，但允许区间重叠，不会出现一个三角形隶属两边的情况

抗锯齿



随机采样边缘点，使其颜色渐变

辐射与渲染方程

利用物理中的辐射可以准确度量光照中的物理量

辐射相关概念

- 功率

通常也被称为**辐射通量**、**通量**

指**单位时间内**通过表面或者空间区域的**能量的总量**，有 $\phi = \frac{dQ}{dt}$

- 辐照度

单位时间内到达单位面积的辐射能量，或到达单位面积的辐射通量，也就是**通量对于面积的密度**，有 $E = \frac{d\phi}{dA}$

- **辐出度**

也称为**辐射出射度、辐射度**，用符号M

与**辐照度**相比，它衡量离开表面的通量密度，两者都可以称为**辐射通量密度**

- **立体角**

可以看成是**弧度的三维扩展**，是**度量三维角度的量**，单位为**立体弧度 (sr)**，等于**立体角在单位球上对应的区域的面积**，即 $\omega = \frac{S}{r^2}$

整个球面的立体角为 4π ，半球面的立体角是 2π

- **辐射强度**

通过**单位立体角的辐射通量**，有 $I = \frac{d\Phi}{d\omega}$ ，辐射强度是为了度量**通过一个点的通量的密度**

辐射强度不会随距离变化而变化，不像点光源的辐照度会随距离增大而衰减，这是因为立体角不会随距离变化而变化

- **辐射率**

辐射率度量**从一个微小面积表面出发，射向某个微小方向的通量**

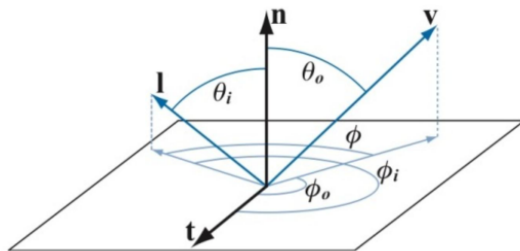
它指**每单位面积每单位立体角的辐射通量密度**，定义为 $L = \frac{d\Phi}{d\omega dA^\perp}$

辐射率**不随距离增加而衰减**，实际上就是我们看到的颜色

- **双向反射分布函数BRDF**

双向反射分布函数BRDF描述**表面入射光和反射光关系**，有

$$f(l, v) = \frac{dL_o(v)}{dE(l)}$$



入射光 l 打到点 p 的一个微小面元，形成**辐照度** E , E 按照一定比例朝 v 方向反射

我们将这个比值定义为入射方向 l ，出射方向 v 的**BRDF值**

郎伯表面的BRDF为 c/π , c 为一个 $[0,1]$ 间的常数

反射方程与渲染方程

- 反射方程

$$L_o(v) = \int_{\Omega} f(l, v) L_i(l) \cos \theta_i d\omega_i$$

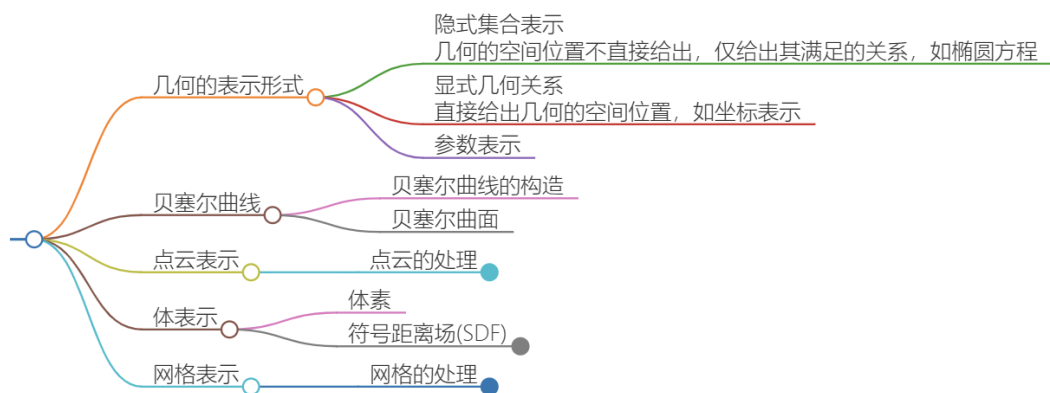
- 渲染方程

$$L_o(v) = L_e + \int_{\Omega} f(l, v) L_i(l) \cos \theta_i d\omega_i$$

其中, L_e 为**自发光**, $f(l,v)$ 为**BRDF值**, $L_i(l)$ 为**入射辐射率**考虑递归

由于需要进行半球面积分, 多次折射之后, 引起指数灾难, 可用蒙特卡洛采样解决

几何



几何的表示形式

- 隐式集合表示

几何的空间位置不直接给出，仅给出其满足的关系，如椭圆方程

- 显式几何关系

直接给出几何的空间位置，如坐标表示

- 参数表示

利用参数方程表示几何曲面不依赖于坐标系的选取，且进行几何变换更容易，交互能力更强

贝塞尔曲线

贝塞尔曲线是一段n次多项式曲线，是构造自由曲线曲面的重要和基本方法之一

贝塞尔曲线利用控制点来控制曲线的基本形状，其中

- 曲线通过第0个和最后一个控制点
- 曲线起始点和终止点处的切线与对应控制点切线一致
- 曲线具有仿射变换不变性

- 曲线具有**凸包性**

贝塞尔曲线的构造

从一次贝塞尔曲线开始

给定两个控制点 P_0, P_1 ，那么**线性贝塞尔曲线**只是一条两点之间的直线，表示为

$$B(t) = P_0 + (P_1 - P_0)t = (1 - t)P_0 + tP_1$$

等同于**线性插值**

二次贝塞尔曲线由三个控制点得到，表示为

$$B(t) = (1 - t)^2 P_0 + 2t(1 - t)P_1 + t^2 P_2, t \in [0, 1]$$

实际上，**任意次贝塞尔曲线**可以被看成是**线性贝塞尔曲线**的嵌套，控制点之间的**线性组合**，有

$$B(t) = \sum_{i=0}^n \binom{n}{i} P_i (1 - t)^{n-i} t^i, t \in [0, 1]$$

用**递归算法**表示，可以得到

$$P_i^k = \begin{cases} P_i, & k = 0 \\ (1 - t)P_i^{k-1} + tP_{i+1}^{k-1}, & k = 1, 2, \dots, n, i = 0, 1, \dots, n - k \end{cases}$$

用**Bernstein基函数** $J_{i,n}(t)$ 定义，可以得到

$$P(t) = \sum_{i=0}^n P_i J_{i,n}(t), 0 \leq t \leq 1$$

其中，Bernstein基函数为

$$J_{i,n}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i}$$

贝塞尔曲面

生成曲面，只需要先生成贝塞尔控制线，再根据控制线上的点生成面即可



EX.

假设有3*3的控制点 $P_{i,j}$ ($i = 0, 1, 2, j = 0, 1, 2$)

首先对**同一行**的点生成贝塞尔曲线，再对**已生成的贝塞尔曲线上的点**再生成贝塞尔曲线

点云表示

点云是空间中点的数据集，可以表示三维形状或对象，通常由**三维扫描仪**获取。

点云中每个点的位置都由一组笛卡尔坐标(X, Y, Z)描述，其主要思想是**双目视觉三角化**



点云的处理

- **点云去噪**

指移除点云数据中的噪声点或异常点，以提高点云数据的质量。

- **点云下采样**

减少点云数据中的点数，从而减小数据规模，提高处理效率

- **估计法向**

计算每个点的法向量，用于描述点云表面的几何特性

- **点云分割**

将点云数据划分为不同的部分，以便于进一步处理和分析

- **点云注册**

将不同视角或不同时间采集到的点云数据**对齐到同一个坐标系中，拼接为一个完整点云**

即求一个**最优的刚体变换**，使得两个点云可以重合

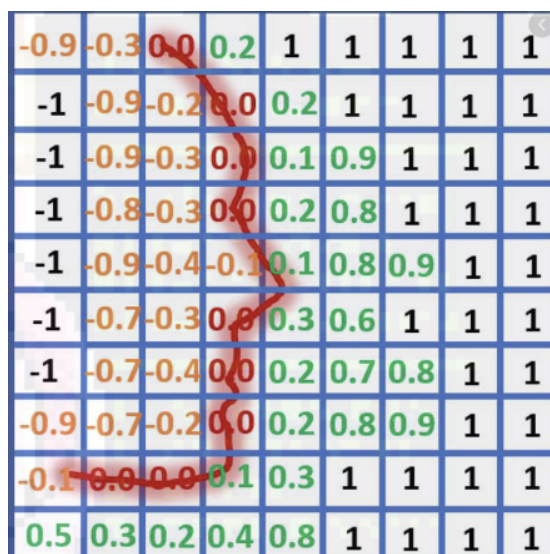
体表示

体素

体素是像素的一种扩展，是三维空间中的一个离散单位，存储**位置、颜色**等信息

符号距离场(SDF)

符号距离场是一种三维场，其中**每个点的值表示该点到最近的表面的距离**，且带符号以表示点在表面内或表面外



- **点云转化为SDF**：对每个体素计算其与点云的最近距离，根据**点云法向判断内外**
- **SDF转化为点云**：找到**临界点进行插值**

符号距离场数据表示规整，与像素类似，便于做卷积等操作

- **数据整体性强**，不容易受噪声干扰
- 适合用于**数据的融合**

但符号距离场数据量大，往往需要**稀疏表示**

网格表示

网格是表示**多面体形状顶点、边、面**的集合，存储格式有PLY,OBJ,STL,OFF等

Triangles			
0	x0	y0	z0
1	x1	y1	z1
2	x2	y2	z2
3	x3	y3	z3
4	x4	y4	z4
5	x5	y5	z5
6	x6	y6	z6
...	...	...	...

三角形直接存顶点

Triangles			
t0	v0	v1	v2
t1	v0	v1	v3
t2	v2	v4	v3
t3	v5	v2	v6
...	...	...	...

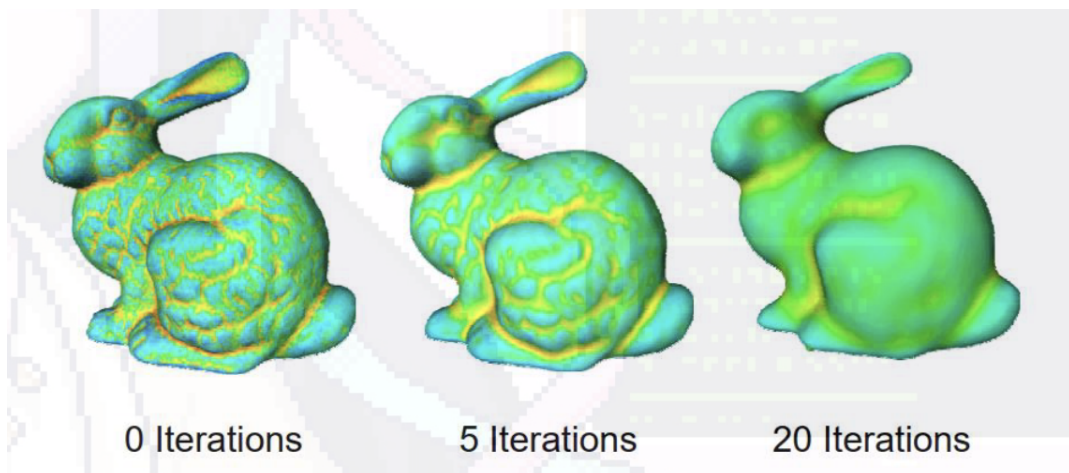
三角形存顶点的索引

Vertices			
v0	x0	y0	z0
v1	x1	y1	z1
v2	x2	y2	z2
v3	x3	y3	z3
v4	x4	y4	z4
v5	x5	y5	z5
v6	x6	y6	z6
...	...	...	...

网格的处理

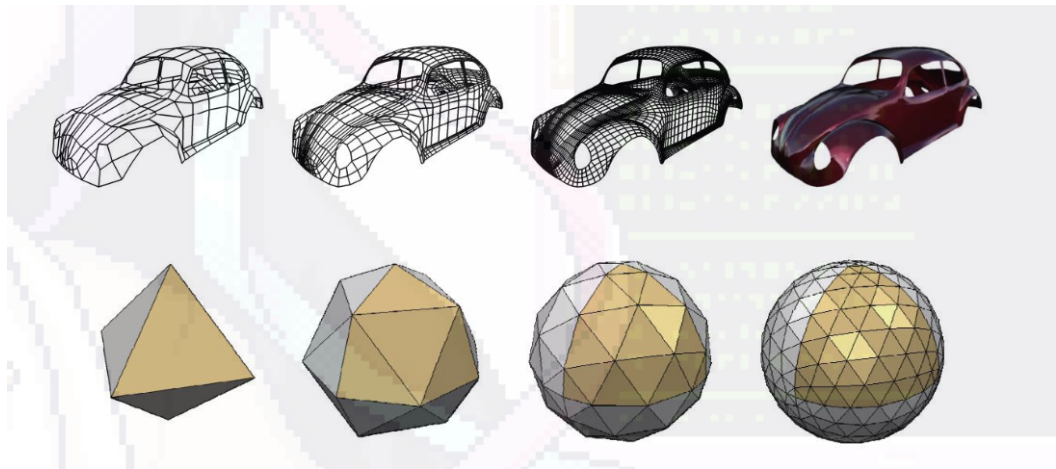
- 网格去噪

利用Laplacian算子，使得曲线的点朝Laplacian算子的方向移动



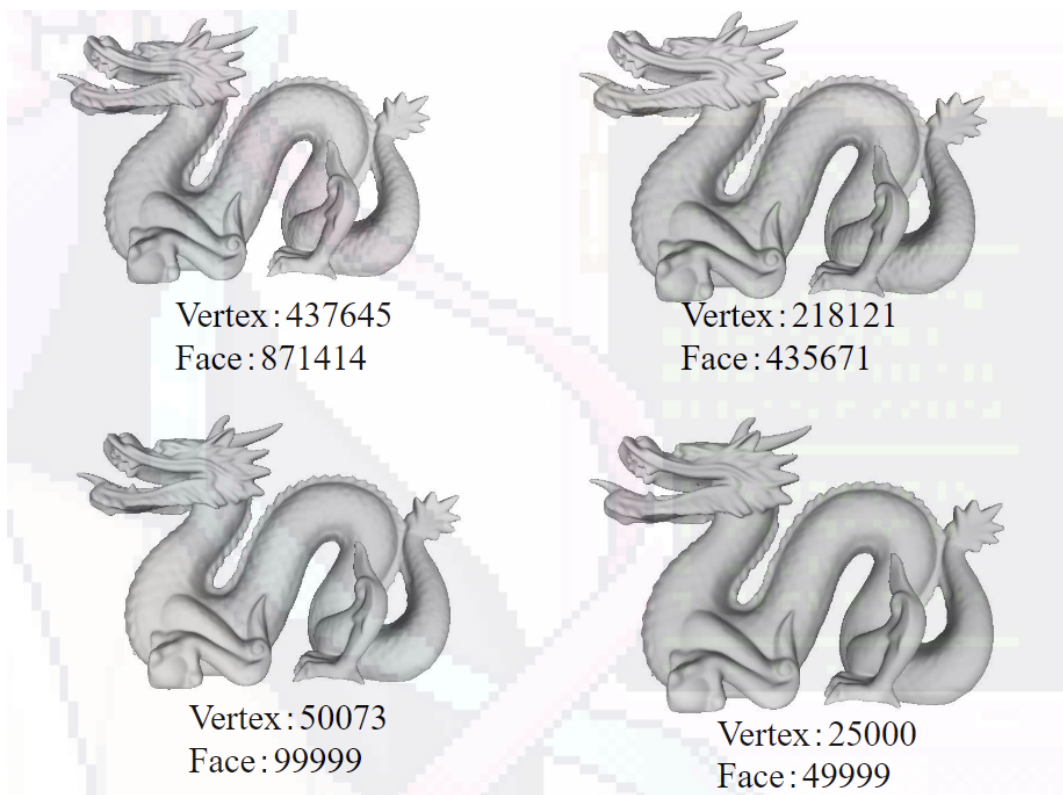
- 网格细分

增加更多控制点，引入更多细节，往往会改变形状



- **网格简化**

将相近的顶点聚为同一点，重新连接边



- **网格正则化**

改变网格的分布

